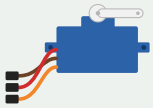
Estacionamento inteligente

Um Raspberry Pi controla as cancelas e um painel web conta quantos carros entraram, saíram e estão dentro. Oito pessoas, oito tarefas.

1 Entre no Raspberry
2 Escolha a sua tarefa
3 Entregue um arquivo

Escolha a sua tarefa

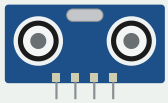


1 Cancelas

2 servomotores

Abre e fecha as cancelas de entrada e de saída.

Entrega `cancelas.py`



2 Presença

Sensor ultrassônico HC-SR04

Diz se há um carro na frente da cancela.

Entrega `presenca.py`



3 Guarita manual

Potenciômetro e botão

O porteiro move a cancela na mão.

Entrega `manual.py`

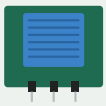


4 Sinalização

LED RGB e buzzer

Cor do estado e musiquinha da cancela.

Entrega `sinais.py`



5 Ambiente

Sensor DHT11

Mede temperatura e umidade do pátio.

Entrega `ambiente.py`



6 Lógica

Python

Decide o que acontece e quando.

Entrega `main.py`



7 Backend

Node.js e SQLite

Guarda e conta entradas e saídas.

Entrega `server.js`



8 Frontend

Página web

Mostra tudo no navegador.

Entrega `public/index.html`

PARA TODOS Faça antes de começar a sua tarefa.

Entrar no Raspberry

O IP muda de rede para rede. O nome `raspberrypi.local` não muda.

1 Descubra o IP de hoje

```
ping raspberrypi.local -4
```

```
POWERSHELL DO SEU PC
PS C:\> ping raspberrypi.local -4

Disparando raspberrypi.local [192.168.0.153] com 32 bytes de dados:
Resposta de 192.168.0.153: bytes=32 tempo=9ms TTL=64
```

O número entre colchetes é o IP. Hoje é 192.168.0.153. Amanhã pode ser outro, então refaça o ping.

▼ **O ping não achou o Raspberry?**

- Confira se o PC e o Raspberry estão no mesmo Wi-Fi ou no mesmo roteador. Rede de escola às vezes bloqueia: use o hotspot do celular nos dois.
- No roteador: rode `ipconfig`, abra o "Gateway Padrão" (ex.: 192.168.0.1) no navegador e procure `raspberrypi` na lista de aparelhos.
- No Raspberry, com teclado e monitor: `hostname -I`.
- Tente direto pelo nome: `ssh dsm@raspberrypi.local`.

2 Entre com SSH

```
ssh dsm@192.168.0.153
```

Troque pelo IP do passo 1. Na primeira vez, digite `yes`. A senha (que não aparece ao digitar) quem passa é o responsável do grupo.

3 Crie a sua pasta e rode um teste

```
DENTRO DO RASPBERRY
dsm@raspberrypi:~ $ cd codigos
dsm@raspberrypi:~/codigos $ mkdir -p task3 && cd task3          # troque 3 pelo número da sua tarefa
dsm@raspberrypi:~/codigos/task3 $ python3 ../rgb_pwm_interativo.py     # Ctrl+C para parar
```

Se o LED RGB da placa mudar de cor, está tudo certo.

Uma pasta por pessoa. **Ctrl+C** ao terminar, senão o pino fica preso.

Fios só com o Pi desligado (`sudo poweroff`).

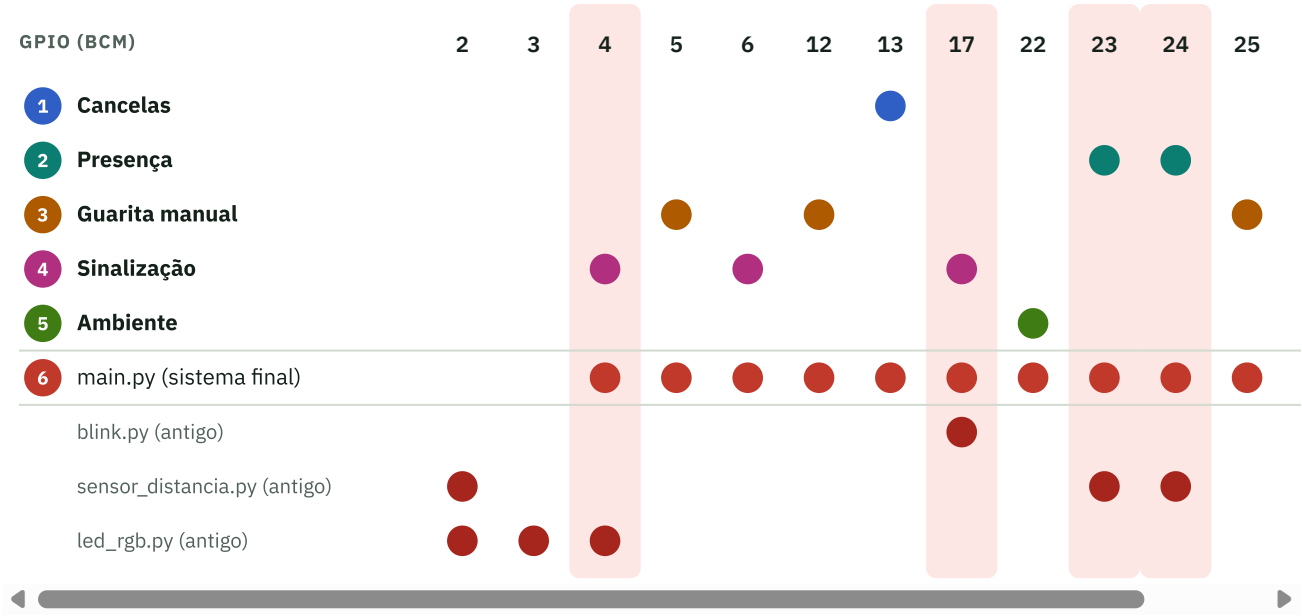
Tarefas 7 e 8 começam no PC e vão para o Pi depois.

▼ **Atalhos: VS Code, copiar arquivos, IP fixo**

- **VS Code:** extensão "Remote - SSH", comando "Connect to Host", `dsm@IP`. Você edita os arquivos direto no Pi.
- **Copiar arquivo do PC para o Pi:** `scp arquivo.py dsm@192.168.0.153:~/codigos/task3/`
- **Parar de caçar IP:** no roteador, reserve um IP fixo para o Raspberry (reserva de DHCP).

Rodar sem conflito

Dois programas só brigam quando usam o mesmo pino. Pinos diferentes convivem bem.



Tarefas 1 a 5 usam pinos diferentes: podem rodar ao mesmo tempo.

main.py usa todos os pinos: só roda com os testes parados.

Scripts antigos (colunas vermelhas) dividem pino com as tarefas: não rode juntos.

> **Se brigarem:** o segundo programa costuma falhar com `GPIO busy`. Com `RPi.GPIO` pode nem avisar: os dois mexem no mesmo pino e o resultado fica estranho. Quase sempre é um programa esquecido rodando. Ache e pare (comandos abaixo).

COMO RODAR

1. Teste rápido, em primeiro plano

```
python3 teste.py
```

O terminal fica preso ao programa. Pare com `Ctrl+C`. É o jeito certo de testar a sua tarefa.

2. Deixar rodando e fechar o SSH

PROGRAMA QUE NÃO PERGUNTA NADA

```
$ nohup python3 -u meu.py > meu.log 2>&1 &
$ tail -f meu.log      # ver os print() (Ctrl+C sai)
$ pkill -f meu.py      # parar
```

PROGRAMA QUE PERGUNTA (INPUT), COMO O CALIBRACAO_POT2.PY

```
$ sudo apt install -y tmux   # uma vez
$ tmux new -s task3          # abre uma sala; rode lá
Ctrl+B, depois D             # sai deixando rodando
$ tmux attach -t task3       # volta à sala
$ tmux kill-session -t task3 # apaga e para o script
```

3. Sistema final: liga sozinho e reinicia se cair

Quando o `main.py` estiver pronto, ele roda como serviço do sistema. Só ele fica dono dos pinos. Os módulos (`cancelas.py`, `presenca.py`...) **não rodam separados**: o `main.py` os importa.

/etc/systemd/system/estacionamento.service

```
[Unit]
Description=Estacionamento inteligente (main.py: cancelas, sensores, LED e buzzer)
After=network.target estacionamento-web.service
Wants=estacionamento-web.service

[Service]
User=dsm
WorkingDirectory=/home/dsm/codigos/final
ExecStart=/usr/bin/python3 -u main.py
Restart=on-failure
RestartSec=3

[Install]
WantedBy=multi-user.target
```

```
$ sudo nano /etc/systemd/system/estacionamento.service   # cole o texto acima; Ctrl+O, Enter, Ctrl+X
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now estacionamento    # liga agora e em todo boot
$ systemctl status estacionamento             # está rodando?
$ journalctl -u estacionamento -f             # ver os print() ao vivo (Ctrl+C sai)
$ sudo systemctl stop estacionamento          # parar para testar um módulo
```

O servidor Node é igual: copie o arquivo, troque o nome para `estacionamento-web` e o `ExecStart` por `/usr/bin/node server.js` (confira com `which node`). Se o DHT11 foi instalado num venv, use `/home/dsm/venv/bin/python` no lugar de `/usr/bin/python3`.

VER E PARAR O QUE ESTÁ RODANDO

```
$ pgrep -af python3                    # quais programas Python estão rodando, e quais arquivos
$ pgrep -af node                       # e quais servidores Node
$ pkill -f meu_script.py               # para um programa pelo nome do arquivo
$ sudo systemctl stop estacionamento   # para o sistema final
```

Cada pessoa testa só os pinos da própria tarefa.

Antes de testar um módulo, pare o serviço do `main.py`.

Servidor Node: só um na porta 3000. `EADDRINUSE` quer dizer que já tem um rodando.

Deixou algo rodando? Avise o grupo, com o nome do script.

AO JUNTAR TODAS AS TAREFAS (FASE 3)

O QUE PODE DAR CONFLITO	REGRA
Duas tarefas criam a mesma peça (por exemplo, o servo 1 em `cancelas.py` e em `manual.py`). Dá erro de pino em uso.	Cada peça tem um só dono: servos em `cancelas.py`, LED e buzzer em `sinais.py`, sensor em `presenca.py`, pot e botão em `manual.py`, DHT11 em `ambiente.py`. Quem precisa de outra peça chama o módulo dono.
Um módulo roda código ao ser importado (um `while True` solto).	Módulo só tem funções. O teste vai dentro de `if __name__ == "__main__":`.
Duas bibliotecas de GPIO no mesmo programa (`RPi.GPIO` do potenciômetro e `gpiozero` das outras peças).	No sistema final, ninguém chama `GPIO.cleanup()` sem argumentos. Use `GPIO.cleanup([12, 25])`. Se der erro na integração, converta o pot para `gpiozero`.
O callback do botão roda em outra linha de execução e mexe na cancela ao mesmo tempo que o loop.	O callback só marca uma flag. Quem mexe na cancela é o loop do `main.py`.
Modo manual ligado e saída pelo painel ao mesmo tempo.	As duas mexem na cancela de saída. Desligue o modo manual (botão) antes de liberar a saída. Enquanto o manual estiver ligado, o sensor de entrada é ignorado.
Funções demoradas (música, leitura do DHT11, medir o pot) param o loop principal.	Músicas curtas. DHT11 só a cada 30 s e fora de abrir ou fechar a cancela. Medir o pot só no modo manual.
`input()` ou arquivo em pasta relativa dentro do serviço (que não tem teclado).	A calibração do pot é feita à mão antes (`python3 manual.py`) e o arquivo fica ao lado do módulo. Sem ele, `posicao_pot()` devolve `None`.

O QUE PODE DAR CONFLITO	REGRA
Os servos tremem quando LED e buzzer usam PWM junto no mesmo programa.	Chame `soltar(qual)` quando a cancela chegar à posição. Se continuar tremendo, confira a alimentação (JP1 e J7).
Hora errada no Raspberry (sem internet): o gráfico da semana sai errado.	Confira com `date` e ajuste o fuso e a hora (veja a tarefa 7).
Ao ligar o Pi, o LED pode acender por um instante e o buzzer apitar baixinho.	É o estado dos pinos antes do programa começar. É normal.

SUA TAREFA Cada pessoa lê só a ficha dela.

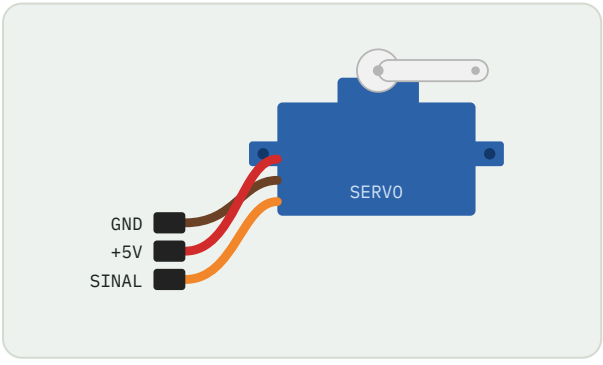
Fichas das tarefas

Cada ficha segue a mesma ordem: entenda, veja a ligação, rode o código, cumpra a missão, entregue.

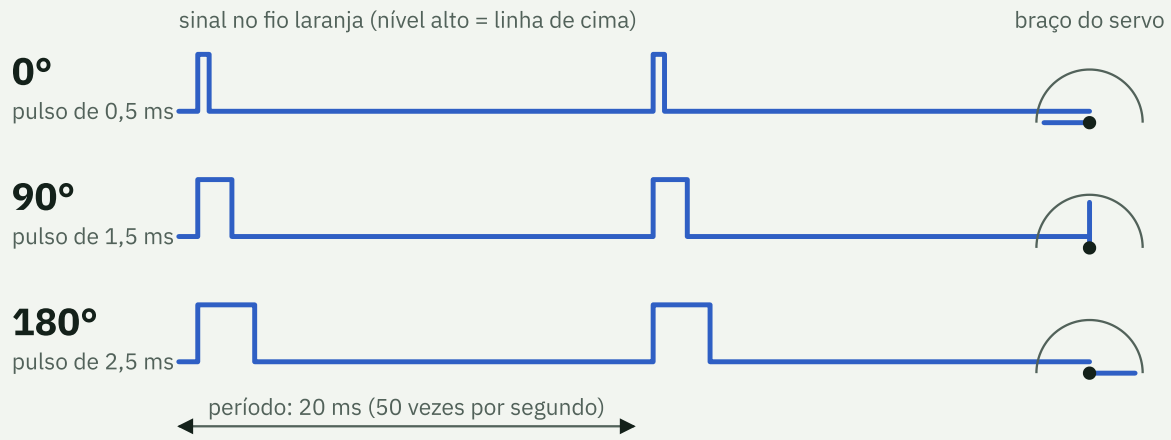
1 Cancelas

Dois servomotores: cancela de entrada (servo 1) e de saída (servo 2).

aprende **PWM** aprende **ângulo do servo**



O servo gira até o ângulo que você mandar. A cancela é o braço dele. A largura de um pulso, repetido 50 vezes por segundo, define o ângulo.



? Como o servo funciona por dentro (saiba mais)

PALAVRAS NOVAS

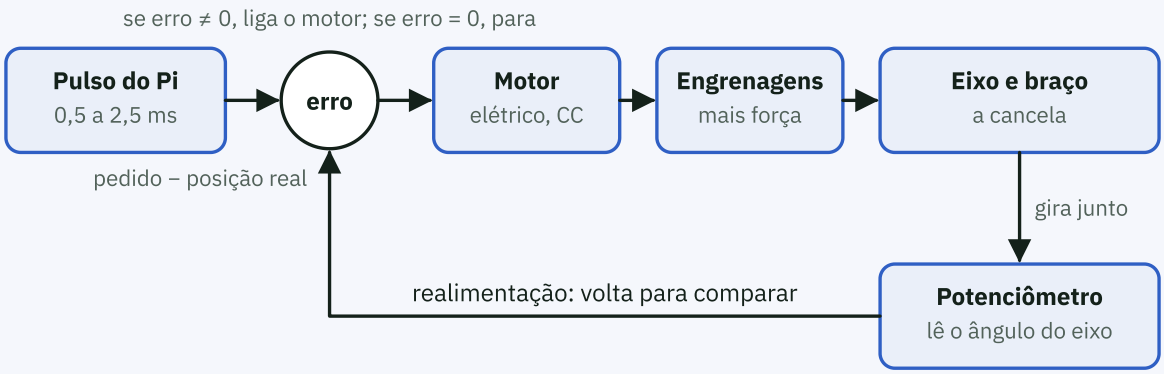
- **Pulso**: um sinal que liga por um instante e desliga, como um toque rápido numa campainha.
- **Realimentação**: olhar o resultado e corrigir, como girar o registro do chuveiro até a água ficar morna.
- **Corrente**: a quantidade de eletricidade passando. Mede-se em ampères (A). 1 mA é um milésimo de ampère.

EM PALAVRAS SIMPLES

O servo é um motorzinho que sabe onde está. Você pede um ângulo, ele gira até chegar e para. Pense no chuveiro: você mexe no registro, sente a temperatura e corrige até ficar certo. O servo faz isso sozinho, muitas vezes por segundo.

O QUE ACONTECE, PASSO A PASSO

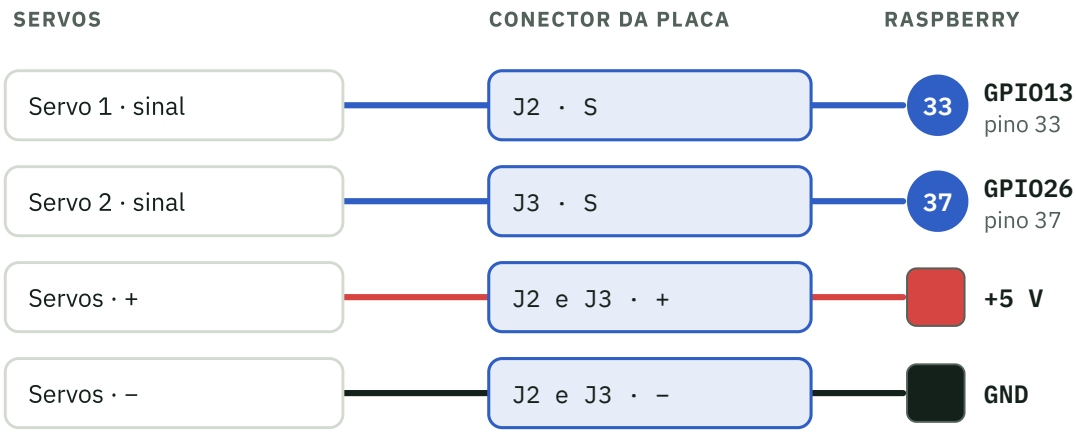
1. O Raspberry manda 50 pulsos por segundo no fio laranja. A **largura** de cada pulso é o pedido: pulso curto, ângulo pequeno; pulso longo, ângulo grande.
2. Uma plaquinha dentro do servo lê esse pedido.
3. Presa ao eixo existe uma peça (um potenciômetro, que você vai conhecer na tarefa 3) que informa o ângulo em que o eixo está de verdade.
4. A plaquinha compara o pedido com o ângulo real. Se são diferentes, liga o motor para o lado certo.
5. Quando ficam iguais, desliga o motor e segura a posição.

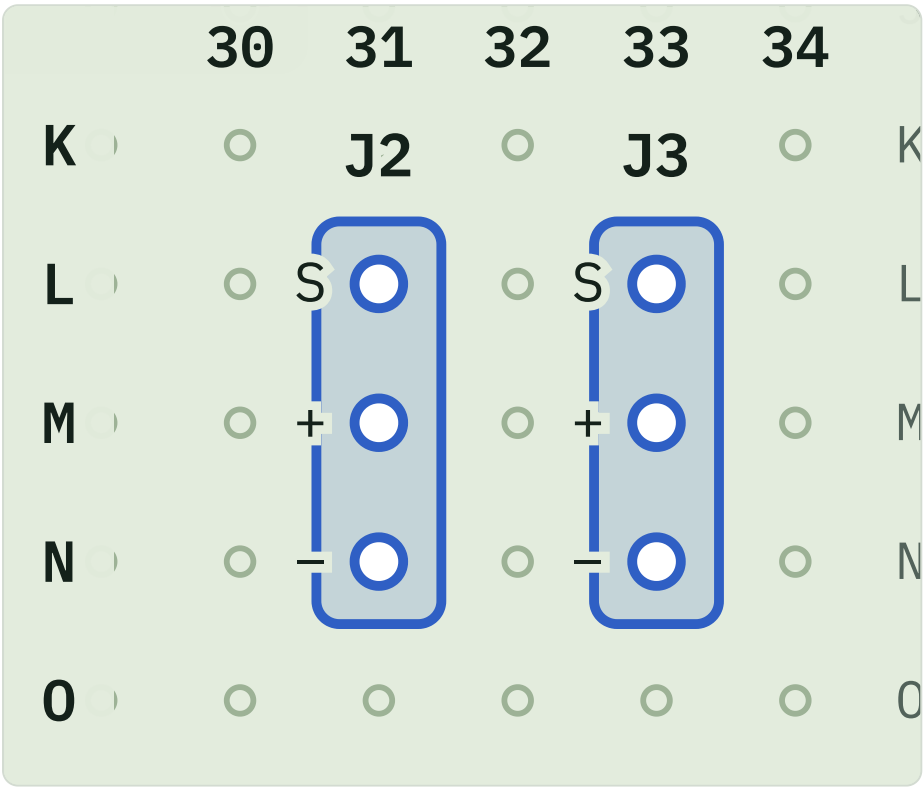


▼ **Quer ir mais fundo? Os detalhes técnicos**

- **Largura vira ângulo.** No nosso código, 0,5 ms é 0°, 1,5 ms é 90° e 2,5 ms é 180° (1 ms é um milésimo de segundo). São 2 ms para 180°, cerca de **11 µs por grau** (1 µs é um milionésimo de segundo).
- **Engrenagens.** Trocam velocidade por força: o motor gira rápido e fraco, e o eixo gira devagar e forte. É por isso que o servo consegue levantar uma cancela.
- **Por que treme.** No Raspberry o pulso é gerado por software, e o sistema operacional atrasa alguns microssegundos de vez em quando. 30 µs de atraso são quase 3°. O código final gera o pulso por software (RPi.GPIO), então esse atraso pode aparecer. O pigpio geraria o pulso por hardware, sem esse atraso, mas o projeto final não o usa.
- **Por que o Pi reinicia.** Parado, o servo gasta pouca corrente. Ao partir ou ao ser forçado, o motor puxa algumas centenas de mA (o SG90 chega perto de 650 mA travado). A fonte de 5 V do Pi cai e ele reinicia. Uma fonte externa resolve.

LIGAÇÃO E LUGAR NA PLACA





J2 (servo 1) e J3 (servo 2). Furos L31–N31 e L33–N33.

CÓDIGO DE PARTIDA

teste_servo.py

```
import RPi.GPIO as GPIO
from time import sleep

# Mesma base do cancelas.py final: PWM de 50 Hz com RPi.GPIO
PINOS = {"entrada": 13, "saida": 26}   # J2 e J3

GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
for pino in PINOS.values():
    GPIO.setup(pino, GPIO.OUT)

pwms = {nome: GPIO.PWM(pino, 50) for nome, pino in PINOS.items()}
for pwm in pwms.values():
    pwm.start(0)

def duty(graus):
    # SG90: pulso de 0,5 a 2,5 ms cobre os 180 graus (20 ms por ciclo)
    pulso_ms = 0.5 + (graus / 180.0) * 2.0
    return pulso_ms / 20.0 * 100.0

def angulo(qual, graus):
    pwms[qual].ChangeDutyCycle(duty(graus))
    sleep(0.5)

angulo("entrada", 0)       # cancela fechada
angulo("entrada", 90)      # cancela aberta
angulo("entrada", 0)
pwms["entrada"].ChangeDutyCycle(0)   # solta o servo: ele para de tremer

for pwm in pwms.values():
    pwm.stop()
GPIO.cleanup(list(PINOS.values()))
```

MISSÃO

- [ ] A entrada fecha em 0° e abre em 90°. A saída é espelhada: fecha em 90° e abre em 0°. Ajuste para o ângulo real.
- [ ] Mova devagar: 5° por vez, com `sleep(0.02)`.
- [ ] Crie o `cancelas.py` com as funções ao lado.

SUA ENTREGA

cancelas.py

`abrir(qual)`

põe a cancela a 90°

`fechar(qual)`

põe a cancela a 0°

`angulo(qual, graus)`

vai para o ângulo pedido, de 0 a 180

`soltar(qual)`

para o sinal do servo (param de tremer). Sem “qual”, para os dois

“qual” é `"entrada"` ou `"saida"`.

Quem usa: **6 Lógica**, **3 Guarita**.

CUIDADO

- **Servo tremendo?** Chame `soltar(qual)` quando a cancela chegar à posição. Se continuar, confira a alimentação (JP1 e J7).
- **Pi reinicia quando o servo mexe?** Jumper JP1 em EXT e fonte no J7.
- **Só o `cancelas.py` cria os servos.** Nenhum outro módulo pode criar PWM nos pinos `13` ou `26`: dois donos da mesma peça dão erro. Guarde o ângulo aberto e o fechado de cada cancela separadamente, porque a montagem pode ser espelhada.

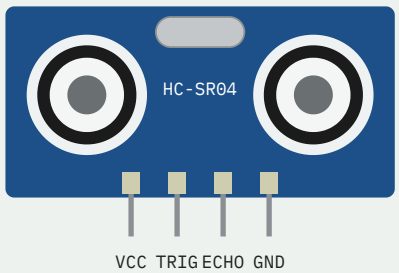
2

Presença

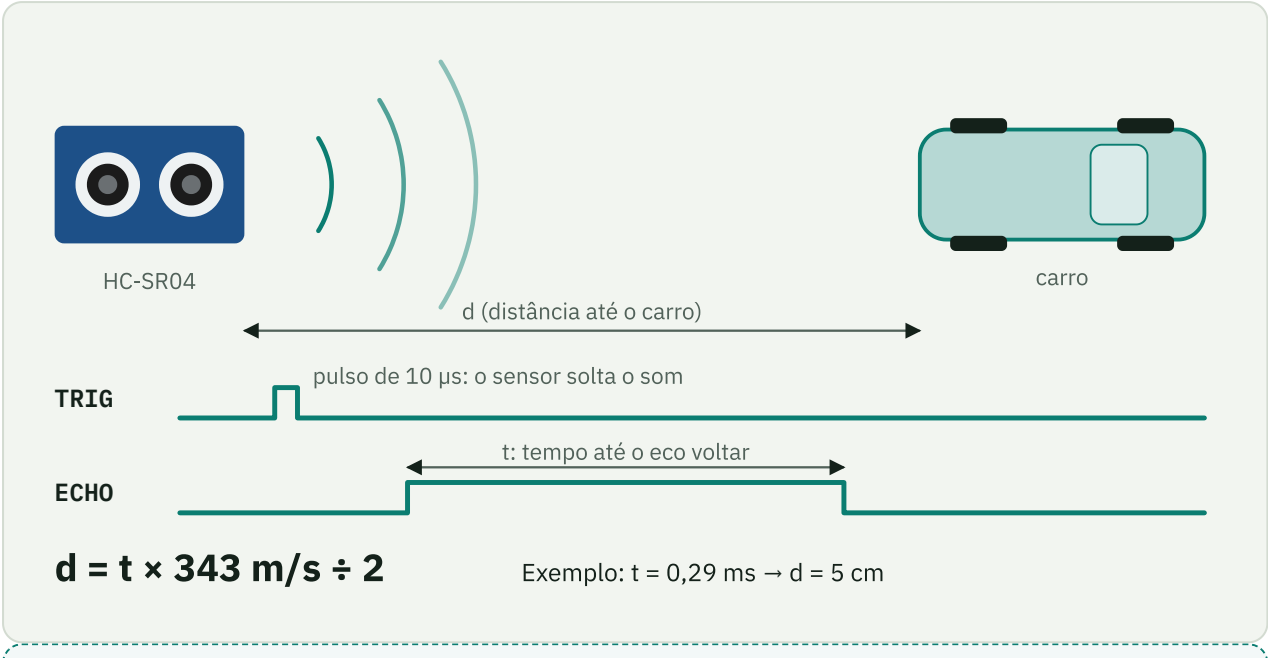
Sensor ultrassônico HC-SR04: tem um carro na frente da cancela?

aprende **medir distância pelo eco**

aprende **divisor de tensão**



O sensor solta um som que a gente não ouve, espera o eco voltar e o tempo vira distância.



Como o ultrassom mede distância (saiba mais)

PALAVRAS NOVAS

- **Eco**: o som que bate num obstáculo e volta, como quando você grita numa montanha.
- **Ultrassom**: um som agudo demais para a gente ouvir. Morcegos e golfinhos usam o mesmo truque.
- **Tensão**: a "força" que empurra a eletricidade, como a pressão da água num cano. Mede-se em volts (V).

EM PALAVRAS SIMPLES

O sensor funciona como um morcego. Ele "grita" num som que não ouvimos, espera o eco voltar e mede quanto tempo demorou. Quanto mais longe o carro, mais demora o eco. Como a velocidade do som é conhecida, o tempo vira distância. É como gritar numa montanha e contar os segundos até ouvir a resposta.

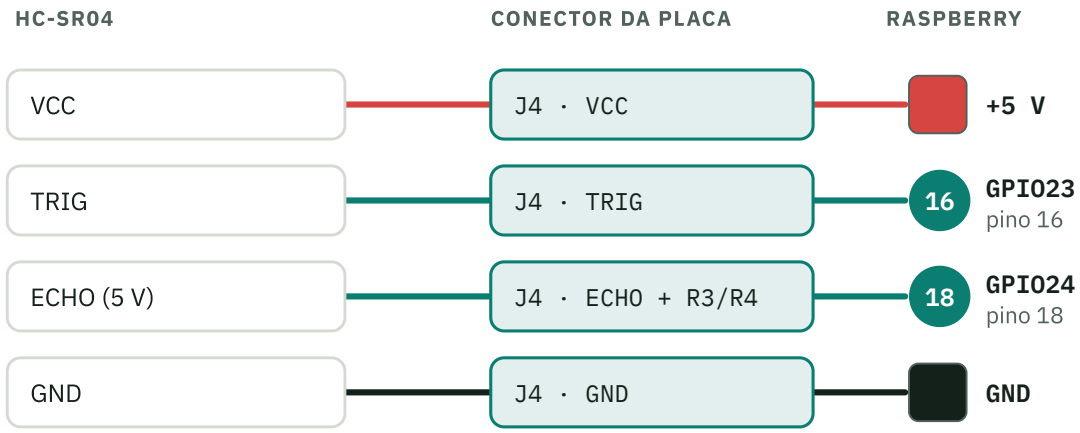
O QUE ACONTECE, PASSO A PASSO

1. O Raspberry manda um pulso curtinho no pino **TRIG**: é o "pode falar".
2. O sensor solta uma rajada de som agudo.
3. O som bate no carro e volta.
4. Quando o eco chega, o sensor levanta o pino **ECHO** e o mantém levantado durante toda a viagem do som.
5. O Raspberry cronometra por quanto tempo o ECHO ficou levantado. Esse tempo é a ida e a volta.
6. Distância = tempo × velocidade do som ÷ 2. Dividimos por 2 porque o som foi e voltou.

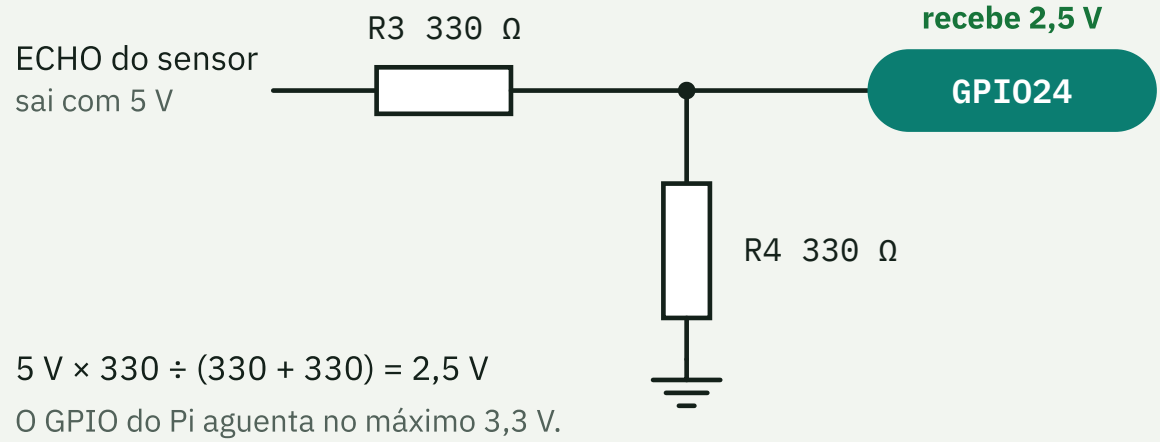
> ▼ **Quer ir mais fundo? Os detalhes técnicos**

- **Os dois “olhos”.** São discos de cerâmica piezoelétrica: um material que vibra quando recebe eletricidade e gera eletricidade quando vibra. Um disco emite (uma rajada de 8 pulsos de 40 kHz, ou seja, 40 mil vibrações por segundo) e o outro escuta. O chip do módulo amplifica o eco, que é minúsculo.
- **Velocidade do som.** No ar, cerca de 343 m/s a 20 °C (`v ≈ 331 + 0,6 × temperatura em °C`). Uma diferença de 10 °C muda a distância medida em uns 1,7%, ou menos de 0,1 cm a 5 cm.
- **Por que falha.** Tecido e esponja absorvem o som. Superfícies inclinadas mandam o eco para o lado e ele não volta. O cone útil do sensor é de uns 15°, e objetos muito finos (menores que uns 9 mm) quase não refletem.
- **Por que o divisor de tensão.** O ECHO sai com 5 V e o pino do Raspberry aguenta no máximo 3,3 V. Dois resistores iguais em série dividem a tensão ao meio, como dois trechos de cano que repartem a pressão: `5 × 330 ÷ (330 + 330) = 2,5 V` .

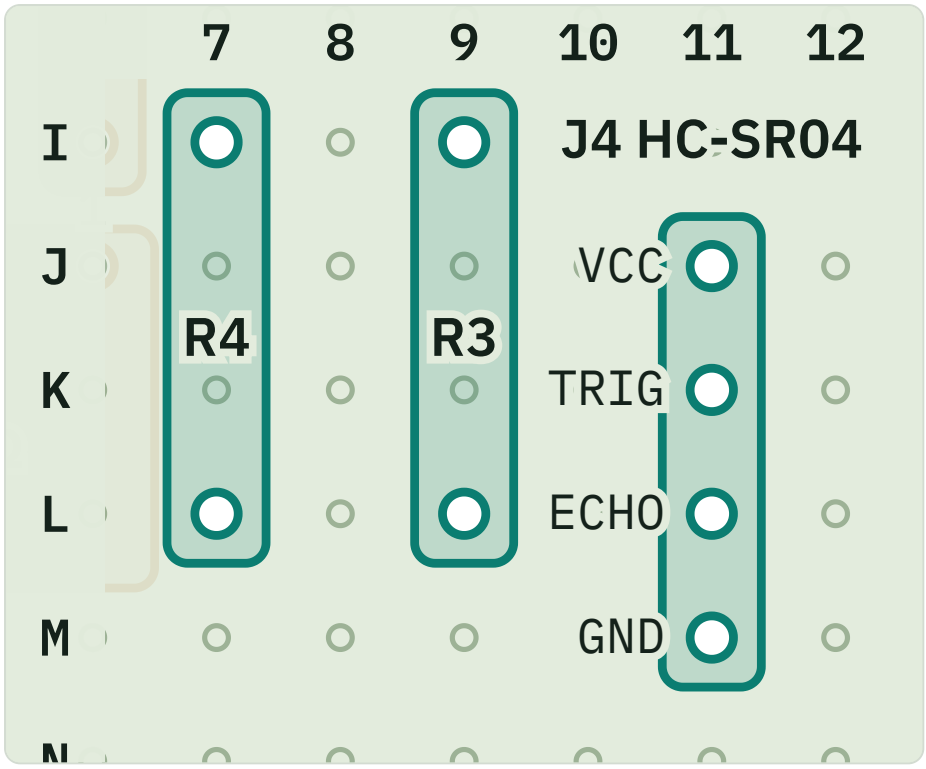
LIGAÇÃO



POR QUE A PLACA TEM O DIVISOR R3 E R4



LUGAR NA PLACA



J4 recebe o sensor (furos J11–M11). R3 e R4 fazem o divisor.

CÓDIGO DE PARTIDA

teste_distancia.py

```
from gpiozero import DistanceSensor
import statistics
from time import sleep

# Mesma configuração do presenca.py do sistema final
sensor = DistanceSensor(echo=24, trigger=23, max_distance=2, queue_len=1)

def distancia_cm():
    # mediana de 5 leituras: tira os picos falsos
    leituras = []
    for _ in range(5):
        leituras.append(sensor.distance * 100)
        sleep(0.02)
    return statistics.median(leituras)

while True:
    print(f"{distancia_cm():5.1f} cm")
    sleep(0.3)
```

MISSÃO

- [ ] Confira com uma régua a 3, 5 e 10 cm. O erro deve ser menor que 1 cm.
- [ ] Use a mediana de 5 leituras para tirar os picos falsos.
- [ ] “Carro presente” = abaixo de 5 cm por 1 segundo seguido.
- [ ] Crie o `presenca.py`.

SUA ENTREGA

`presenca.py`

`distancia_cm()` a distância agora, em cm

`carro_presente()` True se ficou abaixo de 5 cm por 1 s seguido

Quem usa: **6 Lógica**.

CUIDADO

- **Nunca** ligue o ECHO direto no GPIO. Sem o divisor o pino queima.
- Os scripts antigos tinham TRIG e ECHO trocados. Aqui é `echo=24, trigger=23`.
- Tecido e esponja não devolvem o eco. Teste com uma caixa de papelão.
- O HC-SR04 não mede bem abaixo de uns 2 cm. Com o limite em 5 cm, deixe o carrinho parar entre 2 e 5 cm do sensor.

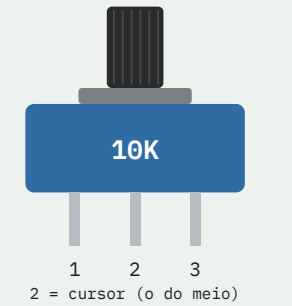
3

Guarita manual

Potenciômetro e botão: o porteiro controla a cancela de saída na mão.

aprende **ler “analógico” com capacitor**

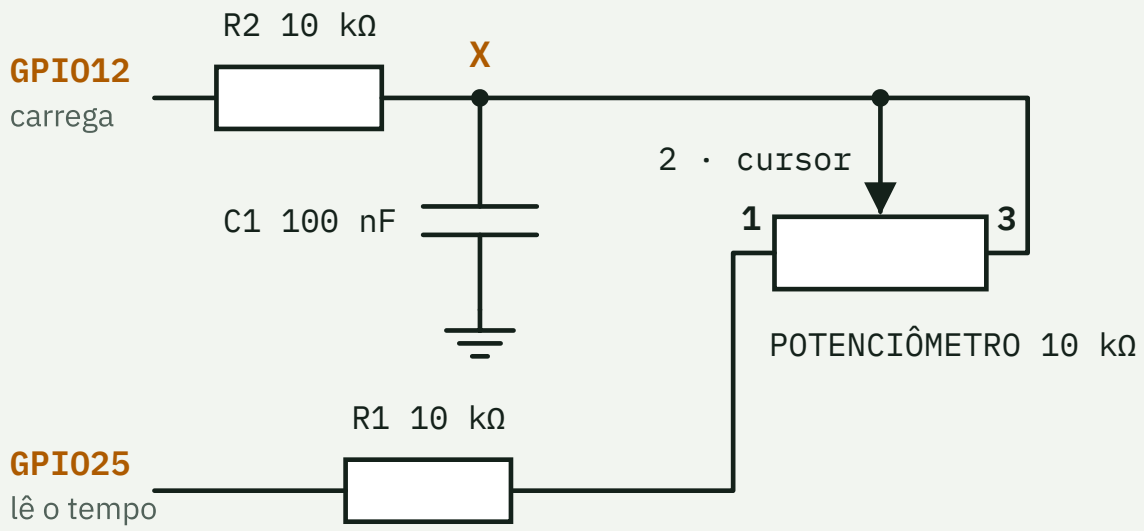
aprende **botão com pull-up**



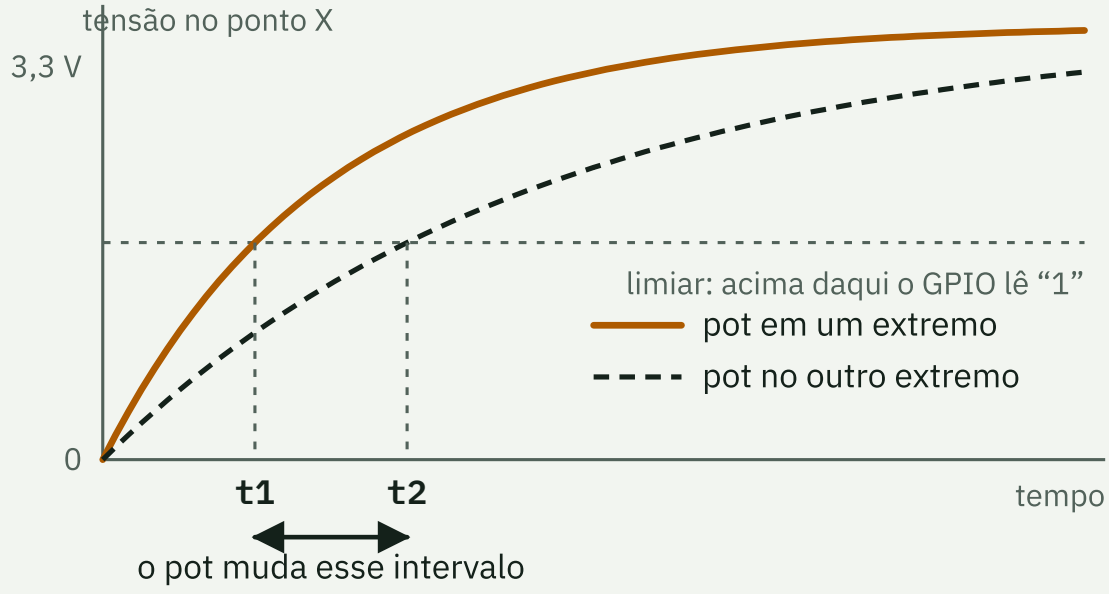


O **potenciômetro** define a **posição** da cancela de saída (0 a 100 vira de fechada a aberta). O **botão** liga e desliga o modo manual.

COMO O POTENCIÔMETRO É LIDO (O PI NÃO TEM ENTRADA ANALÓGICA)



1. O circuito. O ponto **X** é onde se encontram R2, C1, o cursor (2) e a ponta 3. O GPIO12 carrega o capacitor por R2. O GPIO25 lê o resultado por R1 e pela ponta 1 do potenciômetro.



2. O que o código mede. O capacitor carrega aos poucos. O código cronometra quanto tempo a tensão leva para passar do limiar. Girar o potenciômetro muda esse tempo, e o tempo vira a posição de 0 a 100.

Como o potenciômetro e o botão funcionam por dentro (saiba mais)

PALAVRAS NOVAS

- **Resistência**: o quanto um caminho dificulta a passagem da eletricidade, como um cano estreito. Mede-se em ohms (Ω); kΩ é mil ohms.

- **Capacitor**: um pequeno “balde” que enche de eletricidade. Quanto mais estreita a torneira, mais devagar enche.
- **Entrada digital**: um pino que só entende dois valores: 1 (ligado) ou 0 (desligado).
- **Pull-up**: um resistor que “puxa” o pino para 1 quando nada está ligado a ele.

EM PALAVRAS SIMPLES

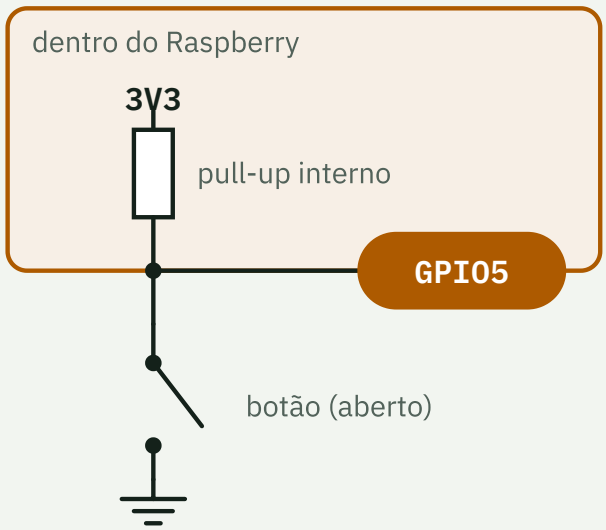
O **potenciômetro** é como o botão de volume do rádio: girando, você muda a dificuldade que a eletricidade encontra. O problema é que o Raspberry só entende ligado ou desligado, e não sabe medir isso direto. O truque é medir o **tempo** que o “balde” (o capacitor) demora para encher. Torneira bem aberta enche rápido, quase fechada demora: o tempo mostra a posição do botão.

O QUE ACONTECE, PASSO A PASSO

1. O código esvazia o capacitor (descarrega).
2. O Raspberry liga o pino GPIO12 e começa o cronômetro. A eletricidade começa a encher o capacitor pelo resistor R2.
3. O pino GPIO25 fica “olhando” a tensão. Quando ela passa de certo nível, ele passa a ler 1.
4. O código para o cronômetro. Esse tempo depende da posição do potenciômetro.
5. Na calibração você mostra os dois extremos (mínimo e máximo). Depois o código converte qualquer tempo em um número de 0 a 100.

> ▼ **Quer ir mais fundo? Os detalhes técnicos**
>
> - **Por dentro do potenciômetro.** É uma trilha resistiva de 10 kΩ com um contato que desliza, o cursor. Entre as pontas a resistência é sempre 10 kΩ. Entre o cursor e uma ponta ela vai de 0 a 10 kΩ, conforme o ângulo.
> - **O capacitor enchendo.** A tensão sobe assim: `V(t) = Vfonte × (1 − e^(−t ÷ RC))`. Aqui `R × C = 10 kΩ × 100 nF = 1 ms`: em 1 ms o capacitor chega a 63% da tensão final.
> - **Do tempo à posição.** Os pinos do Pi decidem 1 quando a tensão passa de mais ou menos a metade de 3,3 V. Nesse limiar, `t ≈ 0,69 × R × C`. A posição do pot muda a resistência do caminho (e quanto da tensão chega ao pino de leitura, junto com um resistor interno de pull-down de dezenas de kΩ), então muda o tempo. Por isso é preciso calibrar: as peças reais variam de 5 a 20% do valor marcado.
> - **Botão e pull-up.** Uma entrada solta “flutua”: capta ruído e lê 0 e 1 ao acaso, como uma antena. O pull-up liga o pino a 3,3 V, então ele lê 1. Ao apertar, o botão liga o pino ao GND (zero). Como o resistor é grande, o GND vence e a entrada lê 0.
> - **Bounce.** Os contatos metálicos quicam por alguns milésimos de segundo ao fechar e geram várias leituras seguidas. O *debounce* ignora as repetições (por exemplo, `bounce_time=0.05`).

COMO O BOTÃO É LIDO



O que o código lê

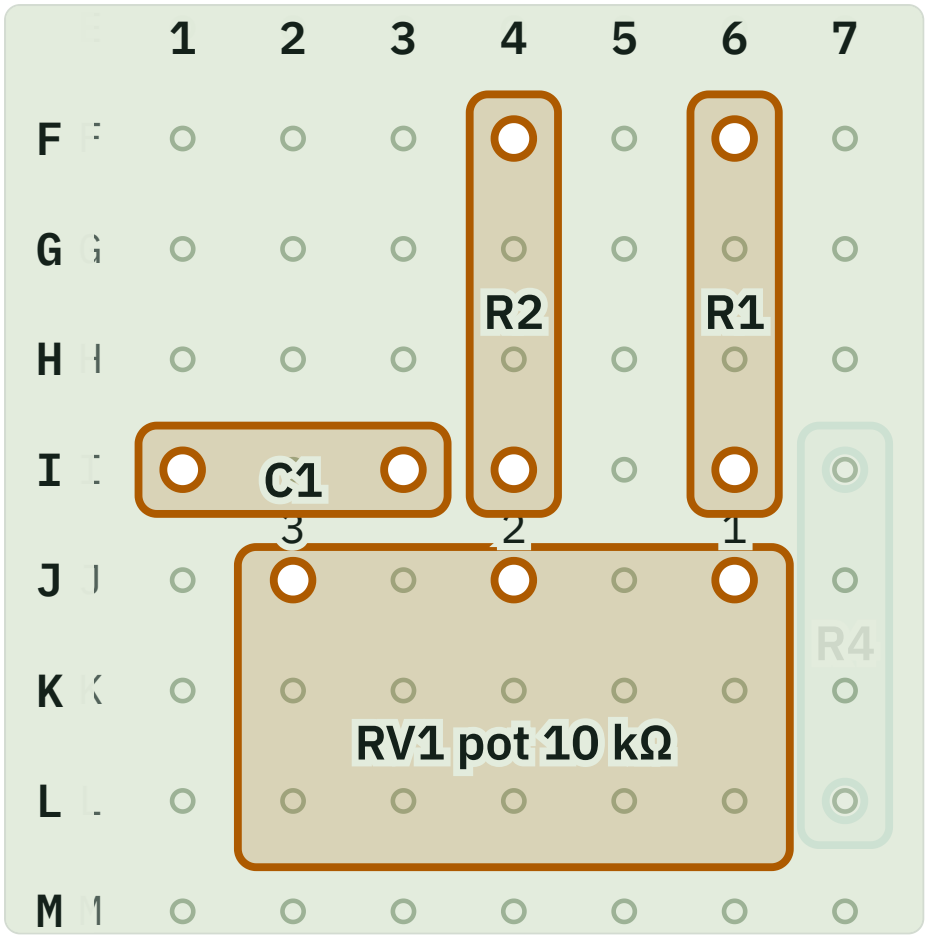
solto — o resistor puxa o pino para cima — **1**

apertado — o botão liga o pino ao GND — **0**

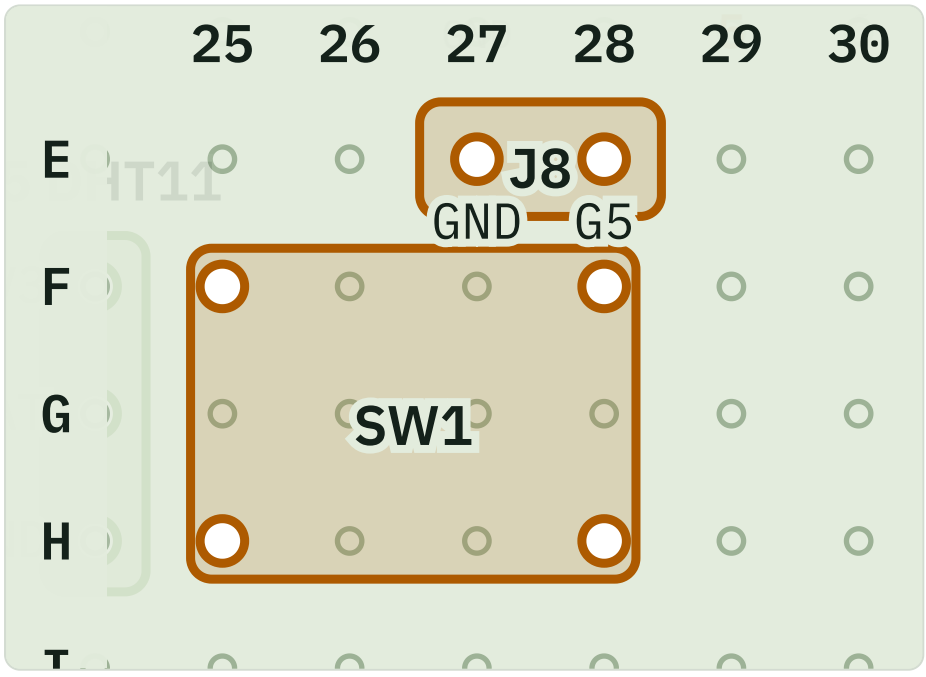
LIGAÇÃO

PEÇA	CONECTOR DA PLACA	RASPBERRY
BOTÃO (SW1)		
SW1 · uma perna	SW1 (ou J8)	29 — **GPIO5** pino 29 · lê o botão
SW1 · perna oposta	SW1 (ou J8)	**GND**
POTENCIÔMETRO (RV1) E CAPACITOR (C1)		
RV1 · cursor, ponta 3, C1	R2 10 kΩ	32 — **GPIO12** pino 32 · carrega
RV1 · ponta 1	R1 10 kΩ	22 — **GPIO25** pino 22 · lê o tempo
C1 · outro lado	direto no GND	**GND**

LUGAR NA PLACA



Potenciômetro RV1, R1, R2 e C1.



Botão SW1 e J8 (botão externo).

CÓDIGO DE PARTIDA: BOTÃO

teste_botao.py

```
from gpiozero import Button
from signal import pause

botao = Button(5)          # pull-up interno: solto = 1, apertado = 0

botao.when_pressed = lambda: print("apertou")
botao.when_released = lambda: print("soltou")

pause()
```

Para o potenciômetro, rode o que já existe: `python3 ../calibracao_pot2.py`.

PRÓXIMO PASSO: GUARDAR A CALIBRAÇÃO

manual.py (pot)

```
# manual.py (esqueleto: você completa os TODO)
# medir() é a função do calibracao_pot2.py: copie-a para cá.
import json
import os

# o arquivo fica ao lado deste módulo, de onde quer que o programa seja chamado
ARQ = os.path.join(os.path.dirname(os.path.abspath(__file__)), "calibracao_pot.json")   # {"min": ms,
"max": ms}

def calibrar():
    # TODO 1: peça "gire para o MÍNIMO", meça com medir(); depois o mesmo para o MÁXIMO
    # TODO 2: grave os dois valores em ARQ com json.dump
    pass

def posicao_pot():
    """Devolve 0 a 100, ou None se a leitura falhou."""
    # TODO 3: leia a calibração de ARQ. Se o arquivo não existir, devolva None.
    #         (calibrar() usa input(), e o serviço do sistema não tem teclado: não a chame aqui)
    # TODO 4: meça o tempo atual com medir()
    # TODO 5: converta para 0..100 com (t - min) / (max - min) * 100
    #         e limite o resultado entre 0 e 100
    pass

if __name__ == "__main__":
    calibrar()          # calibre à mão: python3 manual.py
```

MISSÃO

- [ ] Rode o `calibracao_pot2.py` e veja a posição ir de 0 a 100.
- [ ] Salve o mínimo e o máximo em arquivo, para não recalibrar toda vez.
- [ ] Registre a função do botão com `on_botao(funcao)`.
- [ ] Crie o `manual.py`. Ele só **devolve** a posição e o botão: quem move a cancela é o `main.py`, com `cancelas.angulo()`. Teste a cancela seguindo o pot num script à parte.

SUA ENTREGA

`manual.py`

`posicao_pot()` 0 a 100, ou None se a leitura falhou

`botao_pressionado()` True enquanto o botão está apertado

`on_botao(funcao)` chama a função a cada aperto (só marque uma flag nela)

Quem usa: **6 Lógica**, com **1 Cancelas**.

CUIDADO

- A leitura oscila. Mantenha o filtro (`ALPHA`) e a zona morta do `calibracao_pot2.py` .
- **Não crie o servo aqui.** Se o `manual.py` criar PWM nos pinos 13 ou 26, o `cancelas.py` já é o dono e dá erro quando o `main.py` importar os dois.
- **Callback do botão roda em outra linha de execução.** Dentro dele, só marque uma flag (`apertou = True`). Quem mexe na cancela é o loop do `main.py` .
- **O serviço não tem teclado.** A calibração (que usa `input()`) é feita à mão antes. Sem o arquivo, `posicao_pot()` devolve `None` .
- O pot usa `RPi.GPIO` e o exemplo do botão usa `gpiozero` . Se der conflito, leia o botão com `GPIO.input(5)` , como o `rgb_pwm_interativo.py` faz.

4

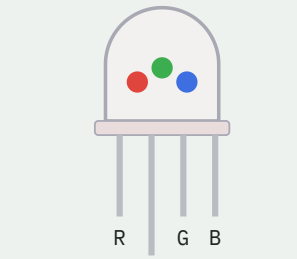
Sinalização

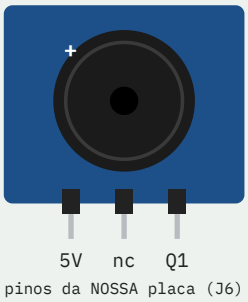
LED RGB e buzzer: a cor do estado e a musiquinha da cancela.

aprende **mistura de cores com PWM**

aprende **LED de ânodo comum**

aprende **notas num buzzer**

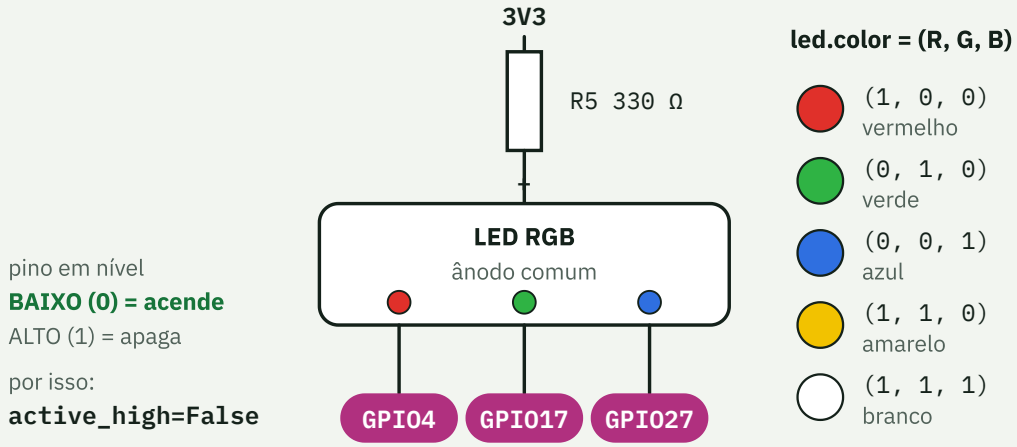




O LED mostra o estado do estacionamento e o buzzer toca a musiquinha quando a cancela abre.

verde: vaga livre amarelo: cancela em movimento vermelho: lotado azul: modo manual

LED RGB: POR QUE “NÍVEL BAIXO ACENDE”



Como o LED RGB e o buzzer funcionam por dentro (saiba mais)

PALAVRAS NOVAS

- **LED**: uma lampadinha de estado sólido: acende quando passa eletricidade num sentido só.
- **Resistor**: uma peça que limita a passagem de eletricidade para proteger as outras.
- **PWM**: ligar e desligar muito rápido para controlar a intensidade, como piscar tão depressa que o olho vê só uma luz mais fraca.
- **Frequência**: quantas vezes por segundo algo vibra. Mede-se em hertz (Hz).
- **Transistor**: um interruptor que é fechado por eletricidade em vez de por um dedo.

EM PALAVRAS SIMPLES

O LED RGB tem três luzinhas dentro de uma só cápsula: vermelha, verde e azul. Misturando as três em intensidades diferentes saem todas as cores, como misturar tintas, só que de luz (o monitor e a TV fazem igual). O buzzer é um alto-falante minúsculo: ao receber um vai-e-vem rápido de eletricidade, a membrana vibra e faz som. Quanto mais rápido vibra, mais agudo é o som.

O QUE ACONTECE, PASSO A PASSO

1. **LED.** Cada cor tem seu pino. Nesta placa o LED é ligado ao 3,3 V pelo outro lado, então o pino em **0** acende e o pino em **1** apaga (como uma gangorra).
2. Para uma cor mais fraca, o pino liga e desliga muito rápido (PWM). O olho não vê o piscar e percebe um brilho médio.
3. Vermelho com verde dá amarelo. As três juntas dão branco.
4. **Buzzer.** O pino do Raspberry é fraco demais para o buzzer. Ele fecha o interruptor eletrônico (o transistor), e o buzzer recebe energia dos 5 V.
5. O código liga e desliga o pino numa frequência. O lá (A4) é 440 vezes por segundo. Cada oitava dobra a frequência (o A5 é 880 Hz).

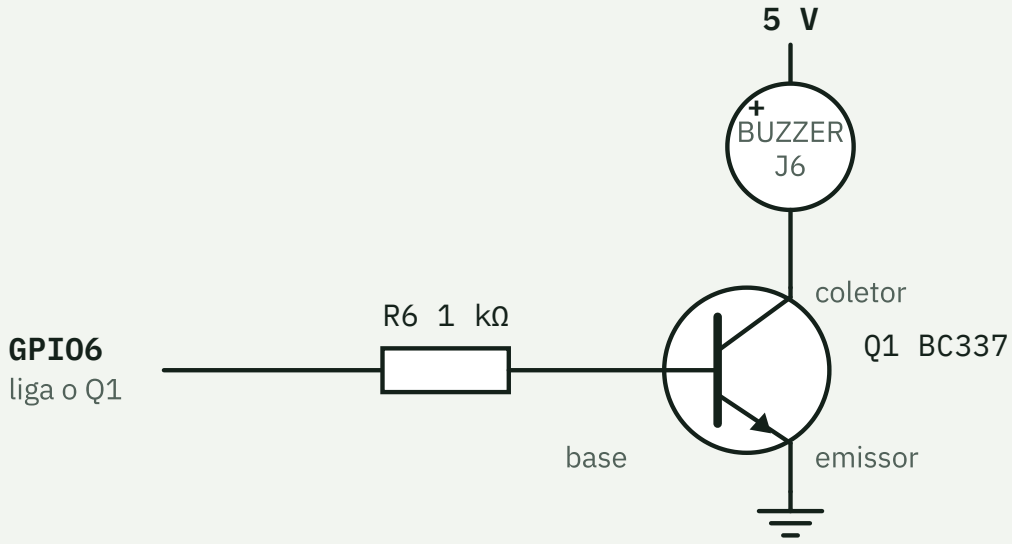
▼ **Quer ir mais fundo? Os detalhes técnicos**

- **Por dentro do LED.** É uma junção de semicondutores: quando a corrente passa, elétrons e lacunas se recombinam e liberam a energia como luz. A energia define

a cor: o vermelho começa a conduzir com uns 2 V, o verde e o azul com uns 3 V.

- **Por que o resistor.** `corrente = (3,3 V − queda do LED) ÷ 330 Ω`. No vermelho: `(3,3 − 2,0) ÷ 330 ≈ 4 mA`. No azul e no verde: `(3,3 − 3,1) ÷ 330 ≈ 0,6 mA`, por isso brilham menos. Sem resistor a corrente não tem limite e o LED queima.
- **PWM.** A proporção do tempo ligado se chama duty cycle. Ligado 50% do tempo parece metade do brilho. Aqui o pino liga e desliga umas 1000 vezes por segundo.
- **Buzzer passivo e ativo.** O passivo é um disco piezoelétrico: toca a nota que você mandar. O ativo tem um oscilador dentro e só toca um tom fixo.
- **O transistor BC337.** Uma corrente pequena na base, `(3,3 − 0,7) ÷ 1 kΩ ≈ 2,6 mA`, deixa passar uma corrente bem maior entre coletor e emissor (ganho de pelo menos 100, ou seja, até uns 260 mA). Os 0,7 V são a queda entre base e emissor.

BUZZER: O TRANSISTOR LIGA O SOM



GPIO6 em nível alto → Q1 conduz → o buzzer recebe 5 V.

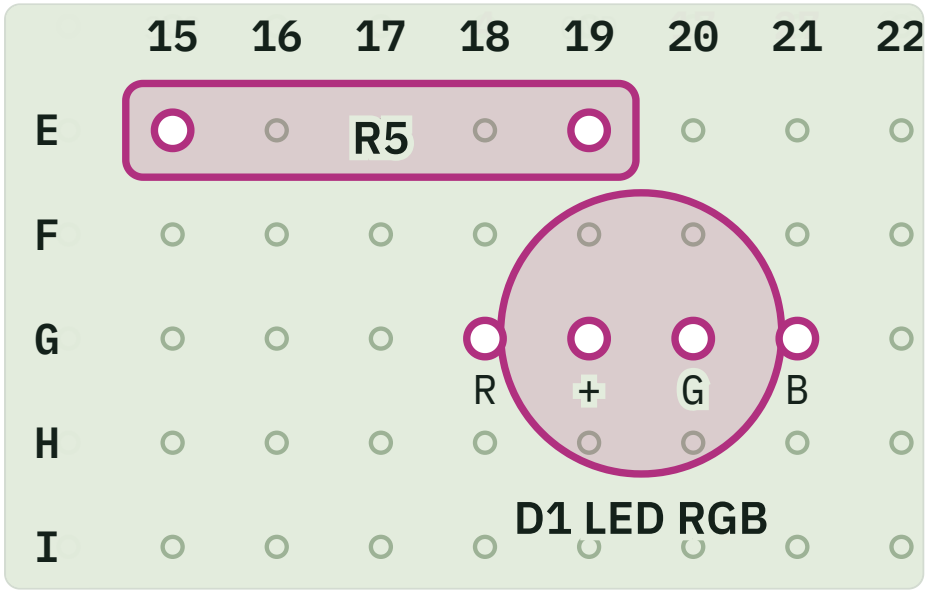



```
Tone("C5") Tone("E5") Tone("G5") Tone("C6")
```

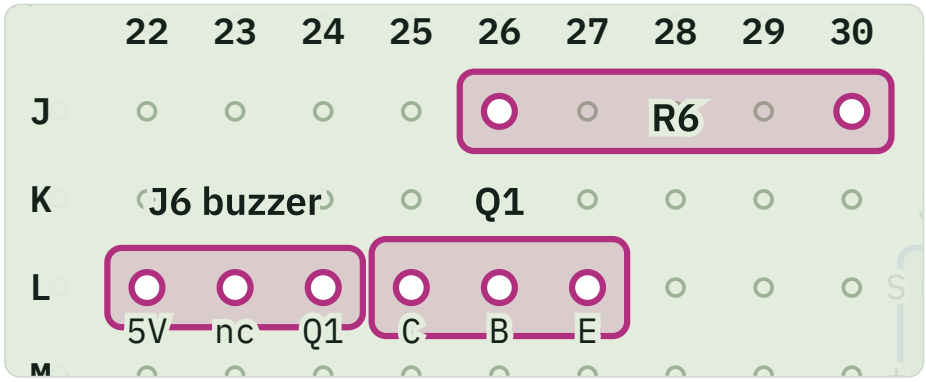
LIGAÇÃO

PEÇA	CONECTOR DA PLACA	RASPBERRY
LED RGB (D1)		
LED · R	D1 · R	7 **GPIO4** pino 7
LED · G	D1 · G	11 **GPIO17** pino 11
LED · B	D1 · B	13 **GPIO27** pino 13
LED · + comum	R5 330 Ω → D1 +	**3,3 V**
BUZZER (J6)		
Buzzer · +	J6 · 5V	**+5 V**
Buzzer · –	J6 · Q1 (transistor)	31 **GPIO6** pino 31 · liga o Q1

LUGAR NA PLACA



LED RGB D1 e o resistor R5.



Buzzer J6, transistor Q1 e resistor R6.

CÓDIGO DE PARTIDA

teste_sinais.py

```
import RPi.GPIO as GPIO
from gpiozero import TonalBuzzer
from gpiozero.tones import Tone
from time import sleep

# Mesma base do sinais.py do sistema final
PINOS_LED = [4, 17, 27]              # R, G, B
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)
GPIO.setup(PINOS_LED, GPIO.OUT, initial=GPIO.HIGH)
pwms = [GPIO.PWM(p, 1000) for p in PINOS_LED]   # PWM de 1000 Hz
for pwm in pwms:
    pwm.start(100)

buz = TonalBuzzer(6, octaves=3)

def cor(r, g, b):
    # brilho de 0 a 100; ânodo comum: nível baixo acende, por isso 100 - brilho
    for pwm, brilho in zip(pwms, (r, g, b)):
        pwm.ChangeDutyCycle(100.0 - brilho)

cor(0, 100, 0)                       # verde: vaga livre
for nota in ("C5", "E5", "G5", "C6"):
    buz.play(Tone(nota))
    sleep(0.2)
    buz.stop()
    sleep(0.03)

cor(0, 0, 0)                         # apagado
for pwm in pwms:
    pwm.stop()
GPIO.cleanup(PINOS_LED)
buz.close()
```

MISSÃO

- [ ] Acenda vermelho, verde, azul, amarelo e branco (o branco do `rgb_pwm_interativo.py` usa 80, 100, 100).
- [ ] Crie 3 músicas: `abrindo` (curta e alegre), `lotado` (2 bips graves) e `alerta`.
- [ ] Faça as cores de estado do quadro acima.
- [ ] Crie o `sinais.py`.

SUA ENTREGA

`sinais.py`

`cor(nome)` acende "verde", "amarelo", "vermelho", "azul" ou "apagado"

`musica(nome)` toca "abrindo", "lotado" ou "alerta"

`bip()` um bip curto

Quem usa: **6 Lógica**.

CUIDADO

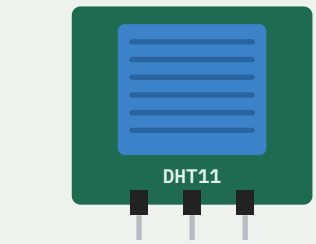
- Verde e azul brilham menos que o vermelho (o LED usa 3,3 V). É esperado.
- **Toca um bip fixo, qualquer nota?** O buzzer é ativo. Use `Buzzer(6).on()` e `.off()` e faça os sinais só com bips curtos e longos.

5

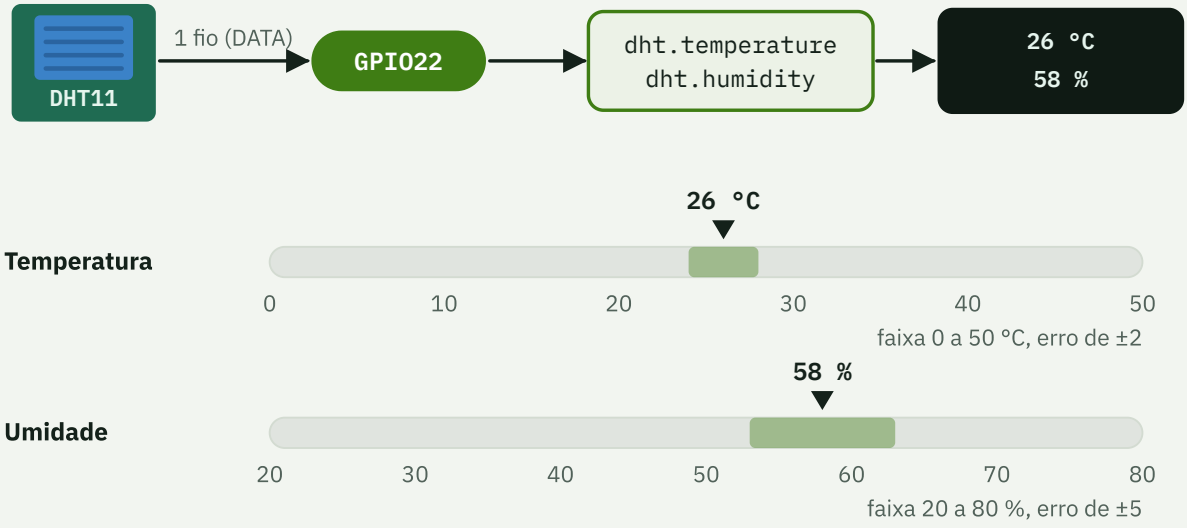
Ambiente

Sensor DHT11: temperatura e umidade do pátio.

- aprende **sensor digital de 1 fio**
- aprende **instalar biblioteca**
- aprende **tratar falhas**



O DHT11 manda temperatura e umidade por um único fio. Cada leitura leva cerca de 1 segundo.



Leia no máximo 1 vez a cada 2 segundos. Falhar de vez em quando é normal.

?Como o DHT11 mede e conversa (saiba mais)

PALAVRAS NOVAS

- **Sensor digital**: um sensor que já entrega o resultado em números, não em tensão.
- **Bit**: a menor informação do computador: 0 ou 1.
- **Termistor**: um resistor cuja resistência muda com a temperatura.

EM PALAVRAS SIMPLES

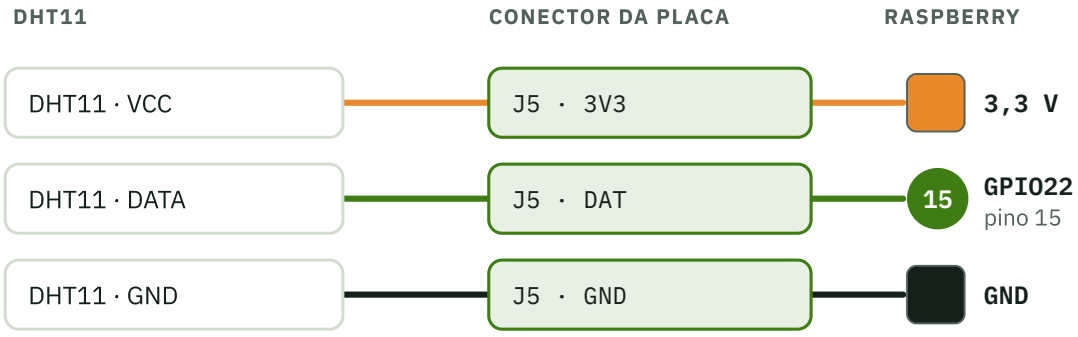
O DHT11 é um termômetro e um medidor de umidade dentro de uma caixinha, junto com um minicérebro. Ele mede sozinho e “conversa” com o Raspberry por um único fio, mandando os dois números. É como um colega que mede e te avisa o resultado batendo pancadinhas curtas e longas, um código combinado entre vocês.

O QUE ACONTECE, PASSO A PASSO

1. O Raspberry “bate na porta”: puxa o fio para baixo por uns 18 ms. É o pedido de medição.
2. O sensor responde e manda **40 bits**, um de cada vez.
3. Cada bit é um pulso: curtinho (uns 27 µs) é 0, mais comprido (uns 70 µs) é 1.
4. Os 40 bits formam 5 números: a umidade (2), a temperatura (2) e uma soma de conferência.
5. O Raspberry confere a soma. Se não fecha, ele descarta a leitura e a biblioteca avisa com um erro (`RuntimeError`). Por isso a gente tenta de novo.

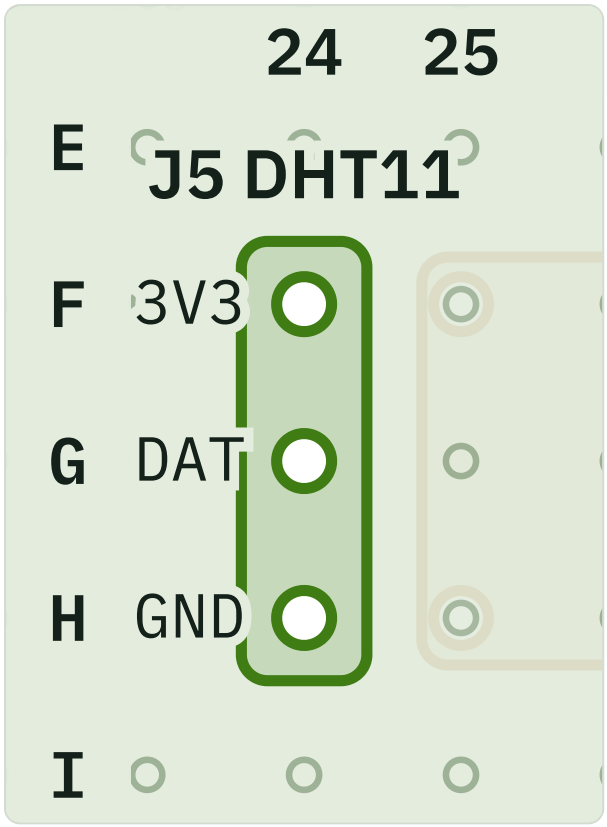
> ▼ **Quer ir mais fundo? Os detalhes técnicos**
>
> - **Temperatura.** Um termistor do tipo NTC: a resistência cai quando esquenta. O chip mede a resistência e converte em °C.
> - **Umidade.** Um polímero que absorve vapor de água. Quanto mais água, menor a resistência elétrica entre dois eletrodos. O chip converte em % de umidade relativa.
> - **Por que o Pi erra.** O Linux não é um sistema de tempo real e pode atrasar a leitura alguns microssegundos, o que estraga um pulso de 27 µs. A soma de conferência não fecha e vem o erro.
> - **Por que 2 segundos.** O DHT11 é lento: precisa de cerca de 1 s por medição. Ler mais rápido só repete erros.

LIGAÇÃO



Confira a ordem antes de encaixar. A placa espera 3V3, DAT, GND (de cima para baixo). Alguns módulos vêm GND, DATA, VCC. Leia a serigrafia do seu. Ao contrário, queima.

LUGAR NA PLACA



J5 recebe o módulo. Furos F24–H24.

INSTALE A BIBLIOTECA (UMA VEZ)

```
sudo apt install -y python3-pip libgpiod2 && pip3 install gpiod
```

O pip respondeu “externally-managed-environment”? Crie um ambiente: `python3 -m venv --system-site-packages ~/venv && source ~/venv/bin/activate` e rode o pip de novo.

CÓDIGO DE PARTIDA

teste_dht11.py

```
"""Lê o DHT11 no GPIO22. Devolve (temperatura, umidade) ou None."""

import time

import gpiod
from gpiod.line import Bias, Direction, Edge, Value

try:
    from pinos import DHT11
except ImportError:
    DHT11 = 22

TENTATIVAS = 5
PAUSA_ENTRE_TENTATIVAS_S = 1.2
LIMIAR_BIT_US = 50.0

def _abrir_chip():
    """Acha o chip de GPIO do Raspberry (o nome muda entre modelos)."""
    for caminho in ("/dev/gpiochip0", "/dev/gpiochip4"):
        try:
            chip = gpiod.Chip(caminho)
        except OSError:
            continue
        if "bcm2835" in chip.get_info().label or "rp1" in chip.get_info().label:
            return chip
        chip.close()
    raise RuntimeError("Chip de GPIO do Raspberry não encontrado")

def _uma_leitura():
    """Faz uma tentativa. Devolve (temperatura, umidade) ou levanta RuntimeError."""
    chip = _abrir_chip()
    try:
        pedido = chip.request_lines(
            consumer="dht11",
            config={DHT11: gpiod.LineSettings(
                direction=Direction.OUTPUT, output_value=Value.INACTIVE)},
            event_buffer_size=256,
        )
    finally:
        chip.close()

    try:
        entrada = {DHT11: gpiod.LineSettings(
            direction=Direction.INPUT, bias=Bias.PULL_UP,
            edge_detection=Edge.BOTH)}
        reconfigurar = pedido.reconfigure_lines
        time.sleep(0.020)
        reconfigurar(entrada)
        eventos = []
        limite = time.monotonic() + 0.25
        while time.monotonic() < limite and len(eventos) < 90:
            if pedido.wait_edge_events(timeout=0.02):
                eventos.extend(pedido.read_edge_events())
    finally:
        pedido.release()

    larguras = []
    subida = None
    for ev in eventos:
        if ev.event_type == ev.Type.RISING_EDGE:
            subida = ev.timestamp_ns
        elif subida is not None:
            larguras.append((ev.timestamp_ns - subida) / 1000.0)
```

```
            subida = None

    bits = [1 if w > LIMIAR_BIT_US else 0 for w in larguras[-40:]]

    # o primeiro bit pode se perder ao soltar a linha; ele é sempre 0 na umidade
    faltam = 40 - len(bits)
    if faltam > 2:
        raise RuntimeError(f"resposta incompleta ({len(bits)} de 40 bits)")
    candidatos = [[0] + [(n >> i) & 1 for i in range(faltam - 1)]
                  for n in range(2 ** max(faltam - 1, 0))] if faltam else [[]]
    erro = None
    for prefixo in candidatos:
        try:
            return _decodificar(prefixo + bits)
        except RuntimeError as e:
            erro = e
    raise erro

def _decodificar(bits):
    """40 bits -> (temperatura, umidade). Levanta RuntimeError se algo não bater."""
    dados = []
    for i in range(5):
        byte = 0
        for b in bits[i * 8:(i + 1) * 8]:
            byte = (byte << 1) | b
        dados.append(byte)
    if (sum(dados[:4]) & 0xFF) != dados[4]:
        raise RuntimeError("soma de conferência não bateu")

    umidade = dados[0] + dados[1] / 10.0
    temperatura = dados[2] + (dados[3] & 0x7F) / 10.0
    if dados[3] & 0x80:
        temperatura = -temperatura
    if not (0 <= umidade <= 100 and -20 <= temperatura <= 60):
        raise RuntimeError(f"valor absurdo ({temperatura} °C, {umidade} %)")
    return temperatura, umidade

def ler():
    """Devolve (temperatura °C, umidade %) ou None se todas as tentativas falharem."""
    for tentativa in range(TENTATIVAS):
        try:
            return _uma_leitura()
        except (RuntimeError, OSError):
            if tentativa < TENTATIVAS - 1:
                time.sleep(PAUSA_ENTRE_TENTATIVAS_S)
    return None

if __name__ == "__main__":
    print(f"Lendo o DHT11 no GPIO{DHT11} (Ctrl+C para parar)...")
    try:
        while True:
            resultado = ler()
            if resultado is None:
                print("sem leitura (confira a ligação do DAT e os 3V3)")
            else:
                print(f"{resultado[0]:.1f} °C | {resultado[1]:.0f} %")
            time.sleep(2)
    except KeyboardInterrupt:
        print("\nTeste encerrado.")
```

MISSÃO

- [ ] Rode o código e veja temperatura e umidade na tela.
- [ ] Sopre perto do sensor: a umidade precisa subir.
- [ ] Crie o `ambiente.py`, que devolve a leitura ou `None`.

☐ Escreva um script de teste que manda uma leitura para `POST /api/ambiente` (a tarefa 6 repete isso a cada 30 s).

SUA ENTREGA

`ambiente.py`

`ler()` devolve (temperatura em °C, umidade em %), ou None se a leitura falhar

Quem usa: **6 Lógica**, **7 Backend**.

CUIDADO

- **RuntimeError de vez em quando é normal.** Por isso o `try/except` e a nova tentativa.
- Não leia mais rápido que a cada 2 segundos.
- **Erro ao ler o DHT11?** Falhar de vez em quando é normal. O `ambiente.ler()` tenta de novo e devolve `None` se todas as tentativas falharem.
- Uma leitura pode levar alguns segundos com as tentativas. No `main.py`, leia só a cada 30 s e nunca no meio de abrir ou fechar a cancela.

6

Lógica

O `main.py` liga tudo: decide quando abrir, avisar e contar.

aprende **máquina de estados**

aprende **juntar módulos dos outros**

aprende **chamar uma API**



Comece sem hardware: crie `cancelas.py`, `presenca.py` e `sinais.py` falsos, que só fazem `print`. O `main.py` já roda com eles. A cada módulo real que chegar, apague o falso.

EntradaCarro 1 s à frente e há vaga: abre. Fecha 2 s depois que o carro passou. Conta uma entrada.

LotadoDentro ≥ vagas (o total vem do servidor): não abre. LED vermelho e a música “lotado”.

SaídaPorteiro clica “Liberar saída” no painel: servo 2 abre, espera 2 s, fecha, conta uma saída.

ManualBotão: liga e desliga o modo manual (LED azul, e o sensor de entrada é ignorado). Pot: a cancela de saída acompanha o potenciômetro. Ao desligar o manual, as duas

cancelas fecham.

Como pensar a lógica do sistema (saiba mais)

PALAVRAS NOVAS

- **Estado**: a situação em que o sistema está agora: livre, abrindo, lotado...
- **Loop**: um trecho do programa que se repete para sempre.
- **Polling**: perguntar de tempos em tempos "mudou alguma coisa?".

EM PALAVRAS SIMPLES

A lógica é o "porteiro automático": um conjunto de regras do tipo "se isso, então aquilo". Pense num semáforo, que passa por verde, amarelo e vermelho, e em cada fase decide o que fazer. O programa lembra em que estado está e, a cada volta, olha os sensores para decidir o próximo passo.

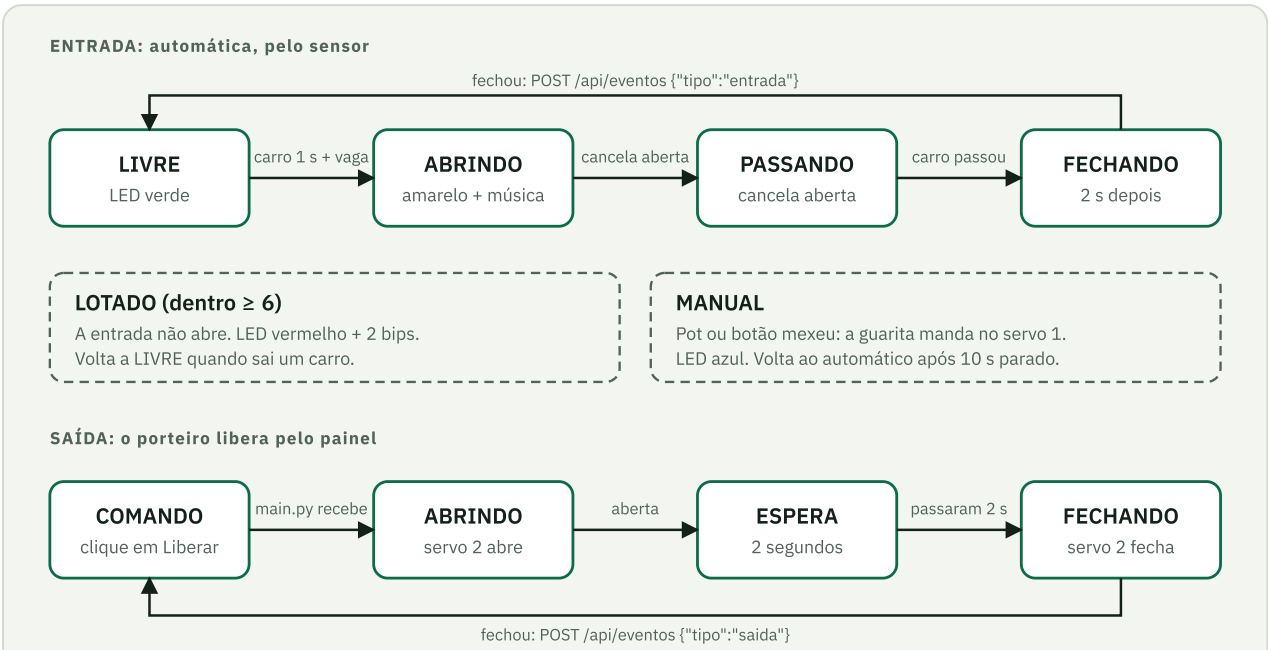
O QUE ACONTECE, PASSO A PASSO

1. A cada volta do loop, o programa olha o sensor: tem carro?
2. Se o carro ficou 1 segundo na frente, o programa pergunta ao servidor se ainda há vaga.
3. Com vaga: acende o LED amarelo, toca a música e abre a cancela.
4. Espera o carro passar, depois mais 2 segundos, e fecha a cancela. Avisa o servidor: "entrou um carro".
5. Volta ao começo e fica esperando o próximo.

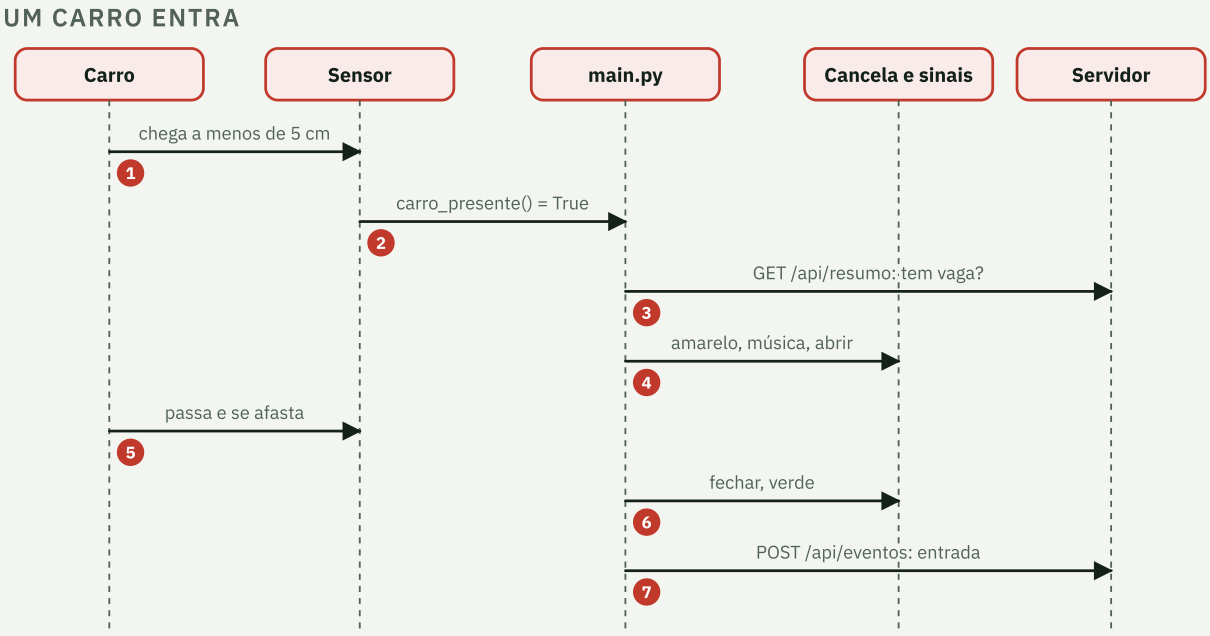
▼ **Quer ir mais fundo? Os detalhes técnicos**

- **A pausa do loop.** O `sleep(0.2)` faz o sistema olhar 5 vezes por segundo. Mais rápido gasta o processador à toa. Mais devagar pode perder um carro rápido.
- **Confirmar pelo tempo.** Exigir 1 s de presença evita falsos avisos (ruído do sensor, um inseto na frente). É a mesma ideia do debounce do botão.
- **Polling.** O `main.py` pergunta ao servidor "tem comando?" a cada volta. É simples e suficiente aqui. Em sistemas maiores o servidor "empurra" o aviso, o que se chama WebSocket.

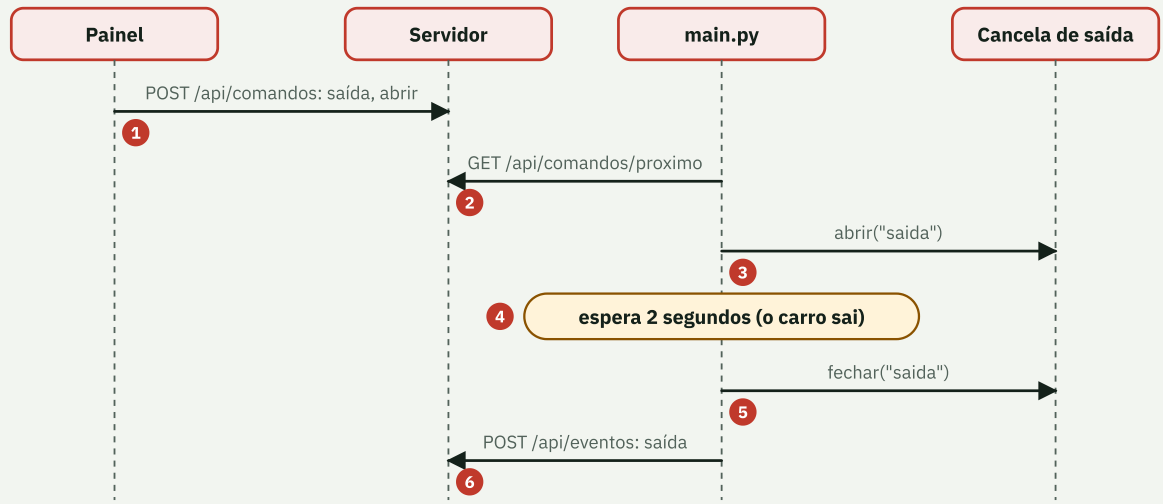
ESTADOS



QUEM FALA COM QUEM



UM CARRO SAI



CÓDIGO DE PARTIDA

main.py

```
# main.py (esqueleto: você completa os TODO)
import time
import requests

import cancelas, presenca, sinais   # no começo use versões "falsas" (só print)

API = "http://localhost:3000/api"
TEMPO_FECHAR = 2      # segundos até a cancela fechar

def resumo():
    return requests.get(f"{API}/resumo", timeout=2).json()    # {"dentro": 3, "vagas": 6, ...}

def registrar(tipo):
    requests.post(f"{API}/eventos", json={"tipo": tipo}, timeout=2)

def liberar_entrada():
    # TODO 1: LED amarelo, música "abrindo" e abrir a cancela de entrada
    # TODO 2: esperar o carro passar (presenca.carro_presente() volta a False)
    # TODO 3: esperar TEMPO_FECHAR, fechar a cancela e registrar("entrada")
    pass

def liberar_saida():
    # TODO 4: abrir a cancela de saída, esperar TEMPO_FECHAR, fechar e registrar("saida")
    pass

while True:
    # TODO 5: o painel pediu "abrir saída"? Chame liberar_saida().
    #         Dica: GET /api/comandos/proximo devolve 204 quando não há comando.
    # TODO 6: há um carro na entrada (presenca.carro_presente())?
    #           se dentro >= vagas: LED vermelho, música "lotado" e NÃO abra
    #           senão: liberar_entrada()
    # TODO 7 (depois): modo manual (manual.py) e leitura do DHT11 a cada 30 s
    #         O manual tem prioridade: enquanto ele estiver ativo, ignore o sensor.
    #         Só o main.py move as cancelas (cancelas.angulo), nunca o manual.py.
    time.sleep(0.2)
```

MISSÃO

- [ ] Mostre os estados desenhados ao time.
- [ ] Rode o `main.py` com módulos falsos.
- [ ] Troque cada falso pelo real conforme as tarefas 1 a 5 entregarem.

- [ ] Implemente o modo manual e o lotado.
- [ ] Se o servidor cair, o Python continua abrindo a cancela.
- [ ] A cada 30 s, leia `ambiente.ler()` e mande `POST /api/ambiente`.

SUA ENTREGA

`main.py`

`roda sozinho` abre e fecha as cancelas, sinaliza, manda entradas, saídas e o ambiente ao servidor

Usa: **1**, **2**, **3**, **4**, **5** e **7**.

CUIDADO

- Falta a biblioteca? `sudo apt install python3-requests`.
- **Os módulos só têm funções.** Um `while True` solto num módulo trava o `main.py` na hora do `import`. Teste dentro de `if __name__ == "__main__":`.
- Músicas e esperas param o loop. Mantenha as músicas curtas. Durante uma entrada, o painel e o manual esperam: é aceitável no protótipo.
- **Batimento.** Envie `POST /api/controlador/batimento` a cada 3 s enquanto espera. Sem ele, o painel mostra o controlador desligado e o botão de saída dá 503.
- **Servidor fora do ar.** Guarde cada entrada ou saída numa fila e envie quando o servidor voltar. A cancela continua abrindo.
- **Cooldown.** Depois de registrar uma entrada, espere 6 s antes de aceitar outro carro na mesma leitura, para não contar duas vezes.
- **Modo manual.** Ao desligar, feche as duas cancelas.
- Feche o contrato da API com a tarefa 7 no primeiro dia.

7

Backend

Node.js com Express e SQLite: guarda entradas e saídas e conta a semana.

aprende **API HTTP**

aprende **SQLite**

aprende **testar com curl**



Recebe do Python cada entrada e saída, grava com data e hora e responde à página quantos entraram, saíram e estão dentro. Comece no seu PC: o Node roda no Windows.

AS TABELAS DO BANCO

eventos

id	INTEGER
tipo	TEXT
criado_em	TEXT

exemplo

```
1 entrada 2026-11-05 08:12
2 saida 2026-11-05 08:40
```

ambiente

id	INTEGER
temperatura	REAL
umidade	REAL
criado_em	TEXT

exemplo

```
1 26.0 58 2026-11-05 08:00
```

comandos

id	INTEGER
alvo	TEXT
acao	TEXT
feito	INTEGER

exemplo

```
1 saida abrir 0
```

eventos: o Python grava cada entrada e saída. ambiente: temperatura e umidade da tarefa 5.
comandos: a página pede “abrir saída”; o Python busca o próximo com feito = 0 e marca feito = 1.

Como funcionam o servidor e o banco (saiba mais)

PALAVRAS NOVAS

- **Servidor**: um programa que fica esperando pedidos pela rede e responde.
- **API**: a lista de pedidos que o servidor entende, como o cardápio de um restaurante.
- **Banco de dados**: um arquivo organizado em tabelas onde os dados ficam guardados.
- **JSON**: um jeito simples de escrever dados em texto, como {"tipo": "entrada"}.

EM PALAVRAS SIMPLES

O servidor é um atendente com um caderno. O Python chega e diz “entrou um carro”. O atendente anota no caderno, com data e hora. Depois o painel pergunta “quantos carros tem?”. O atendente conta as anotações e responde. O caderno é o banco de dados: guarda tudo, mesmo se o Raspberry for desligado.

O QUE ACONTECE, PASSO A PASSO

1. O Python faz um pedido `POST /api/eventos` com `{"tipo": "entrada"}`.
2. O Node confere se o pedido está certo e escreve uma linha nova no banco, com a hora.
3. O painel faz um pedido `GET /api/resumo`.
4. O Node conta as linhas (entradas e saídas) e responde com um pacotinho de dados em JSON.
5. O painel mostra os números na tela.

▼ **Quer ir mais fundo? Os detalhes técnicos**

- **HTTP.** O cliente manda um pedido (método, caminho e corpo) e o servidor devolve uma resposta (código e corpo). `GET` lê e `POST` cria. Códigos: 201 é “criado”, 400 é “pedido inválido”, 204 é “sem conteúdo”.
- **Express.** Liga cada par (método, caminho) a uma função. `app.post('/api/eventos', ...)` diz “quando chegar um POST nesse caminho, rode isto”.

- **SQLite.** Um banco que vive em um único arquivo, sem servidor separado. Ele grava em transações: se a energia cair no meio de uma gravação, o arquivo não fica corrompido.
- **Por que guardar cada evento.** Com a hora de cada um dá para contar por dia, por semana ou por qualquer período, sem perder o histórico.

CÓDIGO DE PARTIDA (JÁ TESTADO: GRAVA, CONTA E SERVE A PÁGINA)

server.js

```
// npm init -y && npm install express better-sqlite3
const express = require('express');
const Database = require('better-sqlite3');

const MAX_VAGAS = 7;                    // fácil de mudar

const db = new Database('estacionamento.db');
db.exec(`
  CREATE TABLE IF NOT EXISTS eventos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    tipo TEXT NOT NULL CHECK (tipo IN ('entrada', 'saida')),
    criado_em TEXT NOT NULL DEFAULT (datetime('now', 'localtime'))
  )`);

const app = express();
app.use(express.json());
app.use(express.static('public'));      // a página da tarefa 8 fica em public/

const conta = (tipo, filtro = '') =>
  db.prepare(`SELECT COUNT(*) AS n FROM eventos WHERE tipo = ? ${filtro}`).get(tipo).n;
const dentro = () => Math.max(0, conta('entrada') - conta('saida'));

app.post('/api/eventos', (req, res) => {
  const { tipo } = req.body;
  if (!['entrada', 'saida'].includes(tipo)) {
    return res.status(400).json({ erro: 'tipo inválido' });
  }
  if (tipo === 'saida' && dentro() === 0) {
    return res.status(409).json({ erro: 'ninguém dentro' });   // evita "dentro" negativo
  }
  db.prepare('INSERT INTO eventos (tipo) VALUES (?)').run(tipo);
  res.status(201).json({ ok: true });
});

app.get('/api/resumo', (req, res) => {
  const semana = "AND criado_em >= datetime('now', 'localtime', '-7 days')";
  res.json({
    entradas: conta('entrada', semana),
    saidas: conta('saida', semana),
    dentro: dentro(),
    vagas: MAX_VAGAS,
  });
});

app.listen(3000, '0.0.0.0', () => console.log('Servidor em http://0.0.0.0:3000'));
```

TESTE SEM O PYTHON

```
POWERSHELL (USE CURL.EXE; CURL SOZINHO É OUTRO COMANDO)PS ...> curl.exe -X POST
http://localhost:3000/api/eventos -H "Content-Type: application/json" -d "{\"tipo\":\"entrada\"}"
PS ...> curl.exe http://localhost:3000/api/resumo
{"entradas":1,"saidas":0,"dentro":1}
```

MISSÃO

- [ ] Rode o código de partida e os dois testes acima.
- [ ] Crie as tabelas `ambiente` e `comandos` e os endpoints do contrato.
- [ ] Some `por_dia` ao `/api/resumo`, para o gráfico.
- [ ] Escreva um `seed.js` com uma semana de dados de teste.
- [ ] Recuse (código 409) uma saída quando não há ninguém dentro. O "dentro" nunca fica negativo. O código de partida já faz isso: entenda por quê.

SUA ENTREGA

`server.js` e **`estacionamento.db`** `todos os endpoints do contrato`

Quem usa: **6**, **5** e **8**.

CUIDADO

- **npm install falhou compilando?** `sudo apt install build-essential python3` e tente de novo. No Pi, confira com `node -v`.
- "Dentro agora" (e as vagas disponíveis, que são `vagas` – `dentro`) usa o total de entradas menos o total de saídas, não só a semana.
- Antes da demo, apague o `estacionamento.db`.
- **Confira a hora do Raspberry.** Sem internet ele pode estar com a hora errada, e o gráfico da semana sai errado. Veja com `date`, ajuste o fuso com `sudo timedatectl set-timezone America/Sao_Paulo` e, se preciso, a hora com `sudo date -s "AAAA-MM-DD HH:MM:SS"`.

8

Frontend

A página que o time e o porteiro vão ver.

aprende **fetch de uma API**

aprende **atualizar a tela sozinha**

aprende **gráfico**

O que o navegador faz por baixo (saiba mais)

PALAVRAS NOVAS

- **Navegador**: o programa que mostra páginas da internet (Chrome, Edge, Firefox).
- **JavaScript**: a linguagem que roda dentro da página e a faz reagir.
- **DOM**: a "árvore" de elementos da página (títulos, números, botões) que o JavaScript consegue mudar.

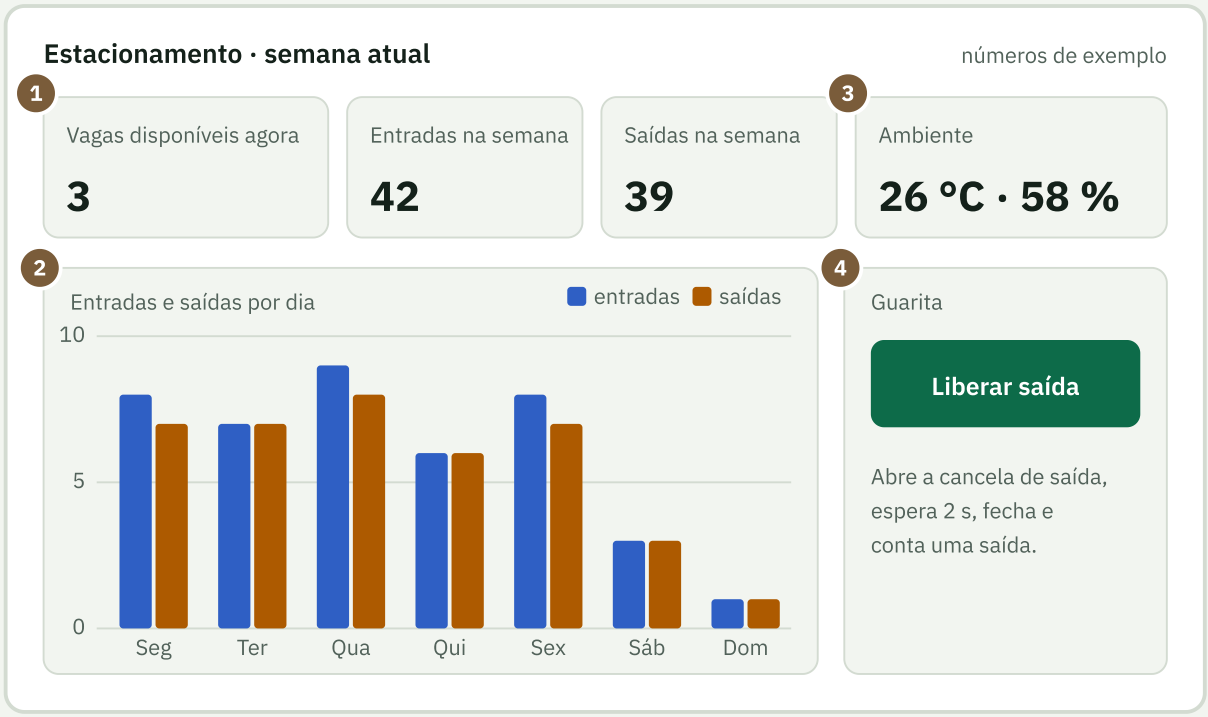
EM PALAVRAS SIMPLES

O painel é a vitrine do sistema: uma página web que mostra os números. Ela não sabe nada sozinha. A cada 2 segundos pergunta ao servidor “quantos carros?”, recebe a resposta e troca os números na tela, sem recarregar a página. É como olhar o placar de um jogo que se atualiza sozinho.

O QUE ACONTECE, PASSO A PASSO

1. O navegador pede a página ao servidor Node e mostra.
2. O JavaScript da página liga um “despertador” que toca a cada 2 segundos.
3. A cada toque, ele pede `/api/resumo` ao servidor.
4. Quando a resposta chega, ele troca o texto dos números e redesenha o gráfico.
5. Se o servidor não responde, a página mostra “sem conexão” em vez de travar.

> ▼ **Quer ir mais fundo? Os detalhes técnicos**
>
> - **Onde o código roda.** O JavaScript da página roda no navegador de quem está olhando, não no Raspberry. O Raspberry só entrega os arquivos e os dados.
> - **fetch e polling.** `fetch` faz o pedido ao servidor. Repetir o pedido de tempos em tempos (polling) é a forma mais simples de manter a tela atualizada.
> - **Gráfico.** O Chart.js desenha dentro de um `<canvas>` a partir de listas de números, no caso as entradas e saídas de cada dia.
> - **Tratar o erro.** Rede e servidor falham de vez em quando. Uma página que avisa é melhor que uma tela travada.



Esboço do painel. Os números são de exemplo.

O QUE CADA ÁREA PEDE À API

1

3 contadores (vagas disponíveis agora, entradas e saídas): `GET /api/resumo` a cada 2 s. Vagas disponíveis = `vagas` − `dentro`.

2

Gráfico da semana (Chart.js): o campo `por_dia` do mesmo resumo.

3

Ambiente: `GET /api/ambiente/ultima`.

4

Liberar saída: `POST /api/comandos` com `{"alvo":"saida","acao":"abrir"}`.

Comece no seu PC, com os dados de teste da tarefa 7 ou com um JSON fixo seu. Salve em `public/index.html` e abra `http://localhost:3000`.

CÓDIGO DE PARTIDA (`public/index.html`)

public/index.html

```
<!doctype html>
<meta charset="utf-8">
<title>Estacionamento</title>
<h1>Estacionamento</h1>
<p>Vagas disponíveis agora: <b id="livres">-</b></p>
<p>Entradas na semana: <b id="entradas">-</b> | Saídas na semana: <b id="saidas">-</b></p>
<script>
async function atualizar() {
  const resposta = await fetch('/api/resumo');
  const d = await resposta.json();
  document.getElementById('livres').textContent = d.vagas - d.dentro;   // vagas disponíveis
  document.getElementById('entradas').textContent = d.entradas;
  document.getElementById('saidas').textContent = d.saidas;
}
atualizar();
setInterval(atualizar, 2000);
</script>
```

MISSÃO

- [ ] Mostre os 3 contadores lendo `/api/resumo`.
- [ ] Adicione o gráfico da semana com Chart.js.
- [ ] Mostre temperatura e umidade.
- [ ] Faça o botão “Liberar saída” chamar a API.
- [ ] Abra no celular: a página precisa caber na tela.

SUA ENTREGA

`public/index.html` `painel lendo a API de verdade`

Usa: **7 Backend**.

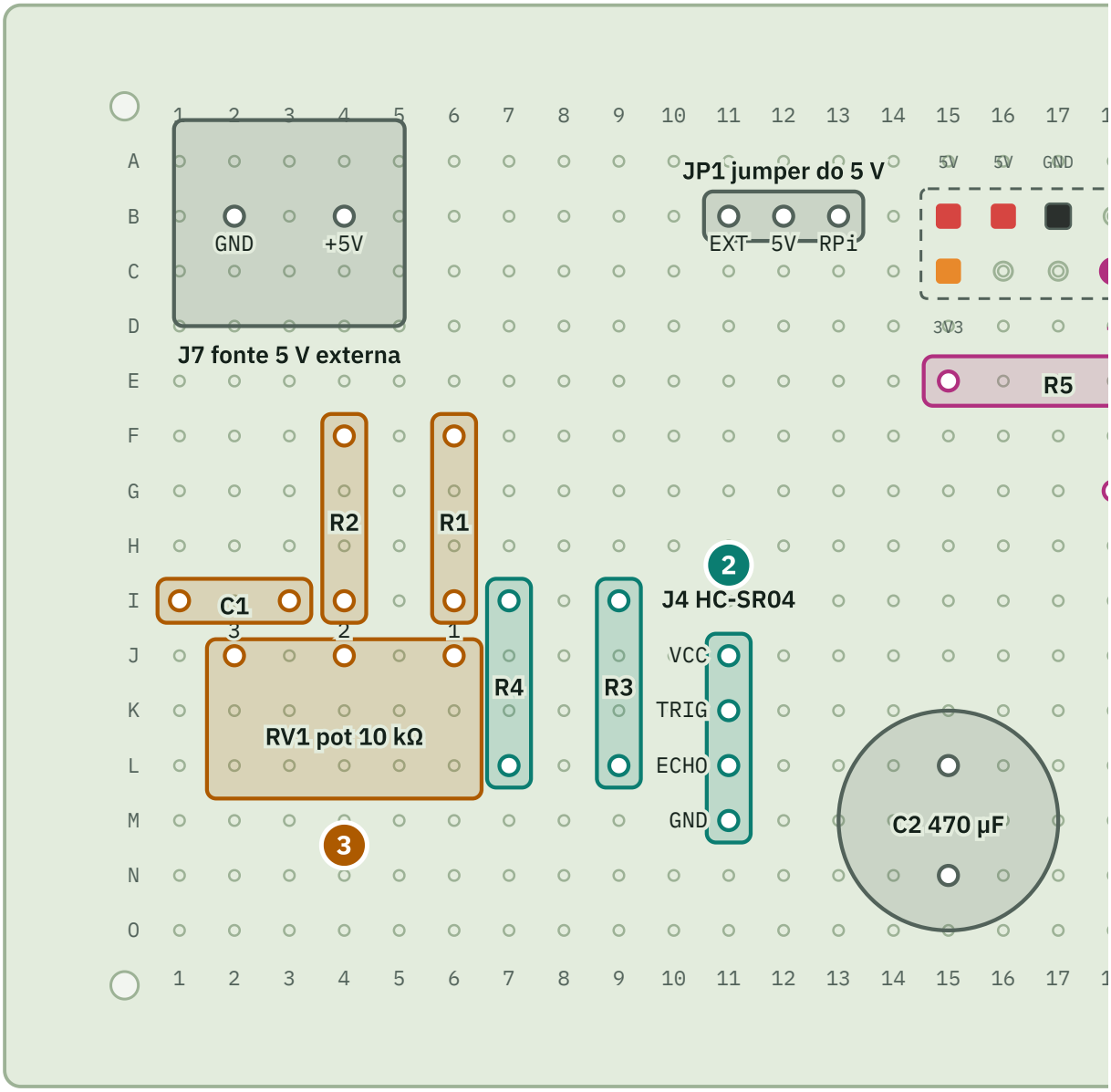
CUIDADO

- No Raspberry, abra `http://IP-do-Pi:3000` (o IP de hoje) ou `http://raspberrypi.local:3000`.
- **Liberar saída.** Resposta 409: ninguém dentro. Resposta 503: o `main.py` está desligado (sem batimento há 15 s). Mostre a mensagem na tela.
- Se a API cair, mostre “sem conexão” em vez de quebrar a tela.

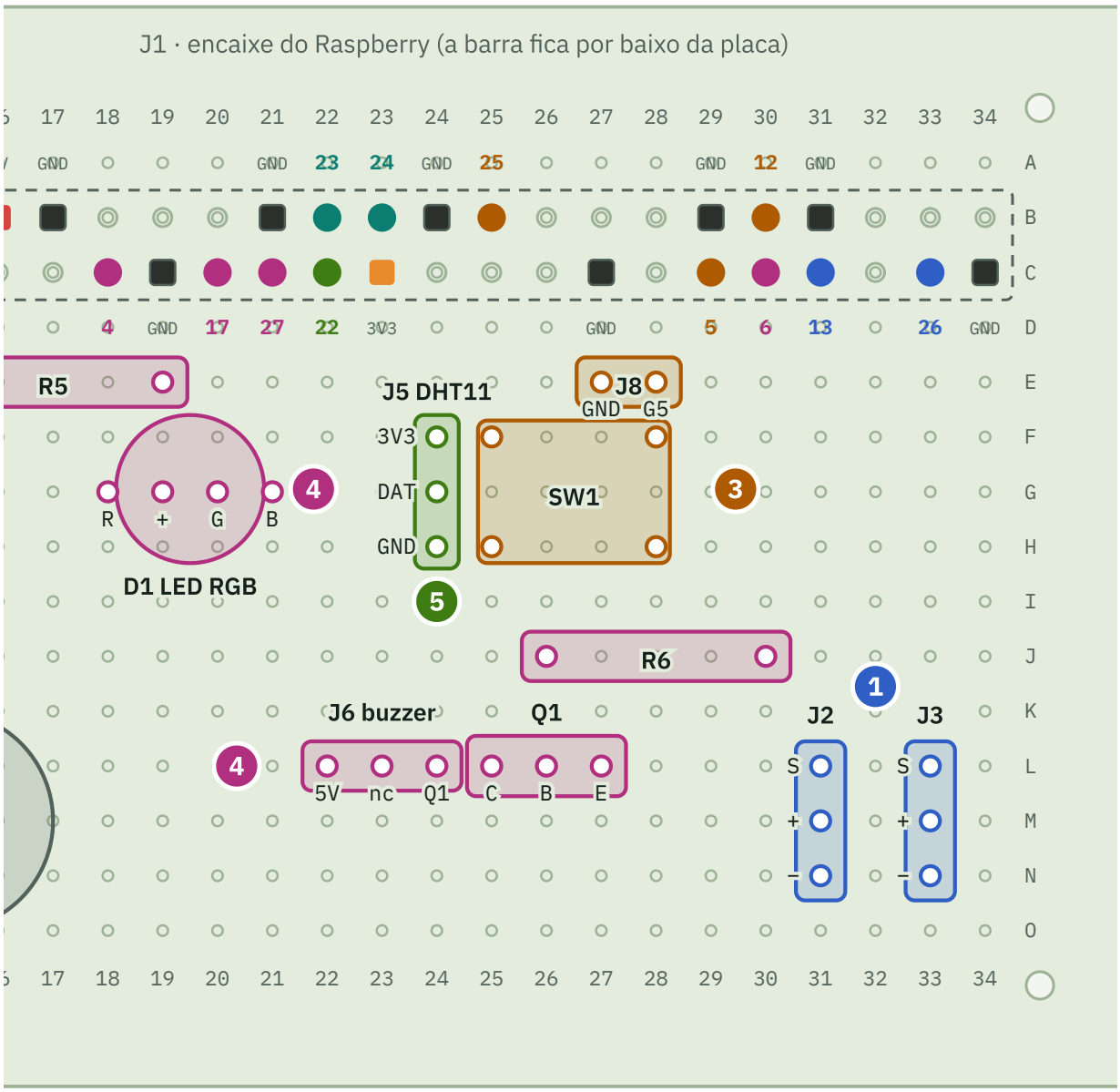
CONSULTA Material de apoio. Não faz parte de nenhuma tarefa: abra só quando precisar.

A placa de interface

A placa ilhada de 5 × 10 cm encaixa por cima do Raspberry. Cada área colorida é de uma tarefa. O mapa vem do projeto da placa, então as posições são as reais.



Metade esquerda (colunas 1 a 17).

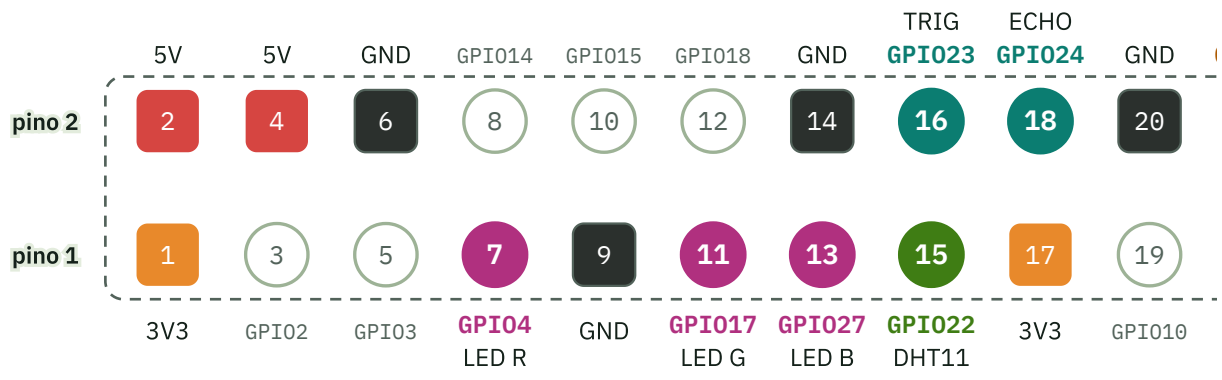


Metade direita (colunas 17 a 34).

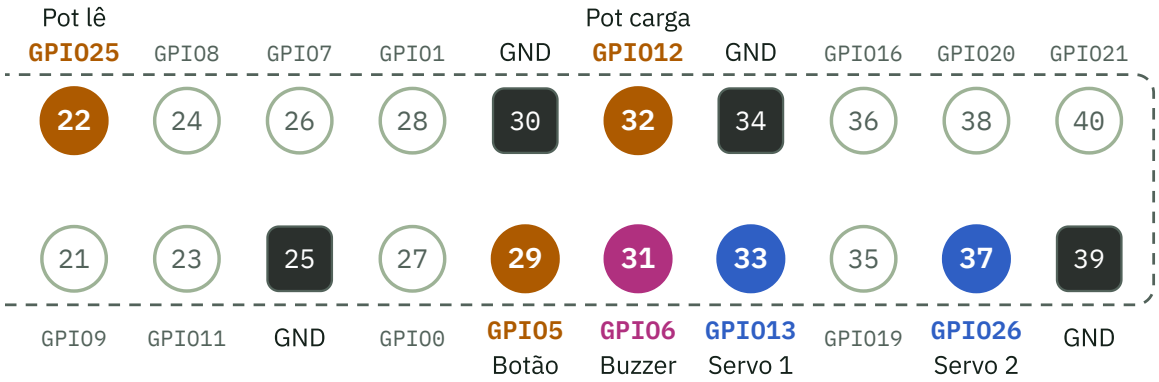
1 Cancelas 2 Presença 3 Guarita manual 4 Sinalização 5 Ambiente
6 Lógica 7 Backend 8 Frontend

Pinos do Raspberry que a placa usa

Número BCM (o de “GPIO”) é o do código Python. O número no círculo é a posição física.



Pinos 1 a 20.



Pinos 21 a 40.

GPIO livre · 5 V · 3,3 V · GND · GPIO em uso (cor da tarefa)

Quem alimenta os servos? O jumper JP1. Em RPi, o 5 V vem do Raspberry. Em EXT, vem da fonte ligada no J7. Pi reiniciando quando o servo mexe: passe para EXT.

Antes de ligar: com o jumper fora, meça continuidade. Não pode haver curto entre os dois 5 V (RPi e EXT), nem entre um 5 V e o GND.

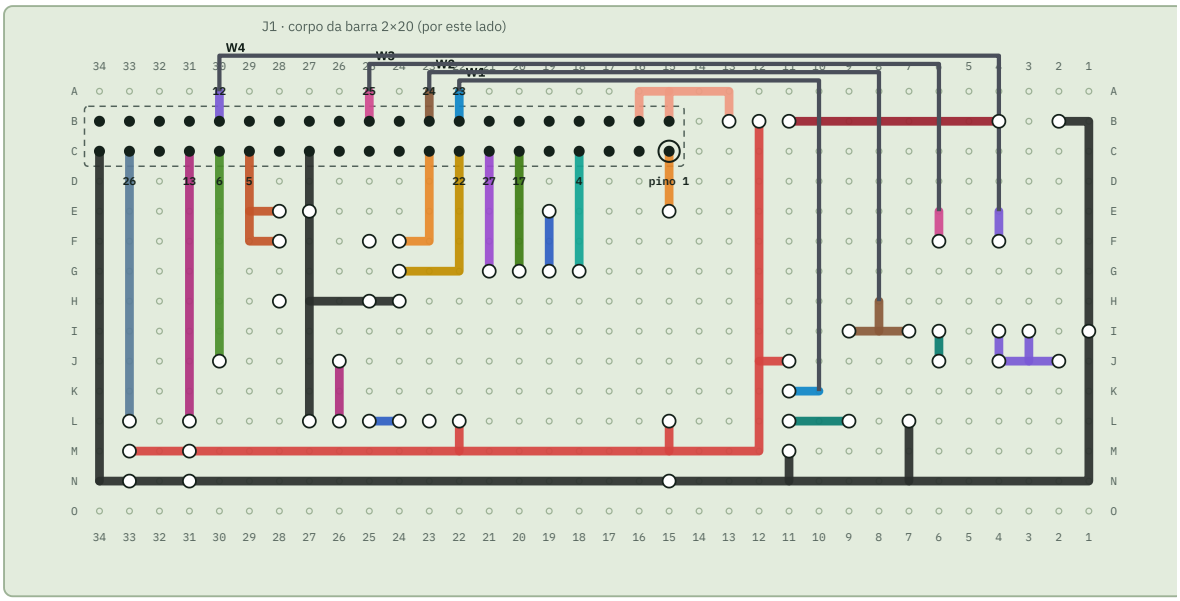
Replicar a placa: lado do cobre, materiais, montagem e testes

O entregável do circuito: o suficiente para montar uma placa igual

O desenho do lado do cobre, o que comprar, o que liga em quê, a ordem de solda e os testes. O projeto do KiCad e a etiqueta Pimaco ficam no repositório,

na pasta `hardware/`.

1. LADO DO COBRE (VIRE A PLACA COMO A PÁGINA DE UM LIVRO: A COLUNA 1 FICA À DIREITA)



Trilhas coloridas por rede: são pontes de solda ou fio estanhado ligando ilhas vizinhas em linha reta. Os fios cinza W1 a W4 são encapados, passam rente à borda de cima e ligam a fileira A ao destino. O retângulo tracejado é o corpo da barra 2 × 20.

+5 V (barramento das cargas) +5 V do Raspberry +5 V da fonte externa 3,3 V
GND fio encapado (W1 a W4) sinais (uma cor por rede)

2. LISTA DE MATERIAIS

QTD	COMPONENTE	VALOR OU MODELO	REFERÊNCIA NA PLACA	OBSERVAÇÃO
1	Placa ilhada 5 × 10 cm	fenolite, face simples, ilhas isoladas, 34 × 15 furos e 4 tiras de cobre na borda	—	furos a cada 2,54 mm
1	Barra fêmea 2 × 20, pinos longos (stacking)	2,54 mm	J1	o corpo fica no lado do cobre e encaixa no Raspberry; os pinos saem por cima

QTD	COMPONENTE	VALOR OU MODELO	REFERÊNCIA NA PLACA	OBSERVAÇÃO
1	Barra de pinos macho 1 × 40 (quebrável)	2,54 mm	JP1 (1×3), J2 e J3 (1×3 cada), J4 (1×4), J8 (1×2)	15 pinos no total. O J4 foi projetado macho: o HC-SR04 tem pinos macho, então ligue com cabo dupont fêmea-fêmea, ou use barra fêmea no lugar
1	Barra fêmea 1 × 40 (quebrável)	2,54 mm	J5 e J6 (1×3 cada)	6 pinos, para os módulos DHT11 e buzzer
1	Jumper (shunt)	2,54 mm	JP1	escolhe a origem do 5 V das cargas
1	Borne de parafuso 2 vias	passo 5,08 mm	J7	entrada da fonte externa de 5 V
3	Resistor	330 Ω, 1/4 W	R3, R4, R5	R3 e R4 formam o divisor do ECHO; R5 limita o LED
2	Resistor	10 kΩ, 1/4 W	R1, R2	circuito do potenciômetro
1	Resistor	1 kΩ, 1/4 W	R6	base do transistor do buzzer
1	Capacitor cerâmico	100 nF	C1	circuito do potenciômetro
1	Capacitor eletrolítico	470 µF, 25 V	C2	marque a polaridade; fica entre o 5 V e o GND
1	Potenciômetro linear	10 kΩ	RV1	
1	Transistor NPN	BC337 (TO-92)	Q1	face plana para você: C, B, E
1	LED RGB 5 mm	ânodo comum	D1	a perna mais comprida é o comum (+)
1	Chave táctil	6 × 6 mm, 4 pinos	SW1	use 2 pinos em diagonal
1	Fio estanhado grosso e fio encapado fino	—	trilhas e W1 a W4	4 pedaços de fio encapado para W1 a W4

QTD	COMPONENTE	VALOR OU MODELO	REFERÊNCIA NA PLACA	OBSERVAÇÃO
PARA USAR A PLACA (NÃO FICAM SOLDADOS NELA)				
1	Raspberry Pi 3 Model B	com cartão microSD e fonte do próprio Pi		
1	Sensor ultrassônico HC-SR04	4 pinos		
2	Servomotor	SG90 ou equivalente (3 fios)		
1	Módulo DHT11	3 pinos, com resistor de pull-up		
1	Buzzer	ligado ao J6 (confira se o seu é ativo ou passivo)		
1	Fonte de 5 V externa (opcional)	para os servos, se o Pi reiniciar quando eles se mexem		

3. ORDEM DE MONTAGEM

1. **Etiqueta.** Imprima a etiqueta Pimaco 6282 a 100% e teste antes em papel comum. Cole a A na borda de cima e a B (cortada na linha tracejada) embaixo. Fure só os círculos, de baixo para cima.
2. **Barramentos.** GND (coluna 1 e fileira N, colunas 34 e 27) e +5V (coluna 12 e fileira M).
3. **Trilhas curtas** de cada bloco: LED, DHT11, botão, buzzer, servos, potenciômetro e HC-SR04.
4. **Pontes das fileiras A e D** até os pinos do J1, antes de soldar a barra 2 × 20.
5. **Barra 2 × 20 de pinos longos**, com o corpo uns 3 mm afastado da placa, e depois os fios W1 a W4.
6. **Componentes** (sugestão: dos mais baixos para os mais altos): resistores, C1, Q1, conectores, RV1, borne J7 e C2.

4. TESTES ANTES DE LIGAR

- Com o jumper do JP1 fora, não pode haver continuidade entre o 5 V do Raspberry e o 5 V externo.
- Nenhum curto entre qualquer 5 V e o GND, nem entre o 3V3 e o GND.
- Com o multímetro em continuidade, confira as trilhas da etiqueta uma a uma.
- Primeira ligação sem os módulos e com o jumper em RPi: meça 5 V no JP1 e 3,3 V no J5.
- Com o jumper em EXT, ligue a fonte externa antes ou junto com o Pi. O – da fonte fica ligado ao GND da placa.

5. ETIQUETA PIMACO 6282

É a folha carta com 2 colunas × 7 linhas de 101,6 × 33,9 mm. A etiqueta A cobre a borda de cima e a B cobre o resto. Imprima a 100% (“tamanho real”, sem “ajustar à página”). O arquivo `etiquetas_teste_papel_comum.pdf` serve para conferir o alinhamento em papel comum, contra a luz sobre a folha. As etiquetas têm 101,6 mm: apare 0,8 mm de cada lado.

▼ **Todas as ligações (cada rede e os pontos que ela liga)**

REDE	PONTOS QUE ELA LIGA (PEÇA.PINO E FURO)
`+5V_RPI`	J1.2, J1.4, JP1.1 (B13)
`+5V_EXT`	J7.1 (B4), JP1.3 (B11)
`+5V`	JP1.2 (B12), C2.1 (L15), J4.1 (J11), J6.1 (L22), J2.2 (M31), J3.2 (M33)
`+3V3`	J1.1, J1.17, R5.1 (E15), J5.1 (F24)
`GND`	J7.2 (B2), C2.2 (N15), C1.2 (I1), J4.4 (M11), R4.2 (L7), J5.3 (H24), SW1.3 (H25), J8.2 (E27), Q1.3 (L27), J2.1 (N31), J3.1 (N33), J1.vários
`BUZ_NEG`	Q1.1 (L25), J6.3 (L24)
`ECHO_5V`	J4.3 (L11), R3.2 (L9)
`GPIO12_POT_LEIT`	J1.32, R2.2 (F4)
`GPIO13_SERVO1`	J1.33, J2.3 (L31)
`GPIO17_LED_G`	J1.11, D1.3 (G20)
`GPIO22_DHT`	J1.15, J5.2 (G24)
`GPIO23_TRIG`	J1.16, J4.2 (K11)

REDE	PONTOS QUE ELA LIGA (PEÇA.PINO E FURO)
`GPIO24_ECHO`	J1.18, R3.1 (I9), R4.1 (I7)
`GPIO25_POT_CARGA`	J1.22, R1.2 (F6)
`GPIO26_SERVO2`	J1.37, J3.3 (L33)
`GPIO27_LED_B`	J1.13, D1.4 (G21)
`GPIO4_LED_R`	J1.7, D1.1 (G18)
`GPIO5_BOTAO`	J1.29, SW1.2 (F28), J8.1 (E28)
`GPIO6_BUZZER`	J1.31, R6.1 (J30)
`LED_A`	R5.2 (E19), D1.2 (G19)
`POT_1`	RV1.1 (J6), R1.1 (I6)
`POT_X`	RV1.2 (J4), RV1.3 (J2), C1.1 (I3), R2.1 (I4)
`Q1_BASE`	R6.2 (J26), Q1.2 (L26)

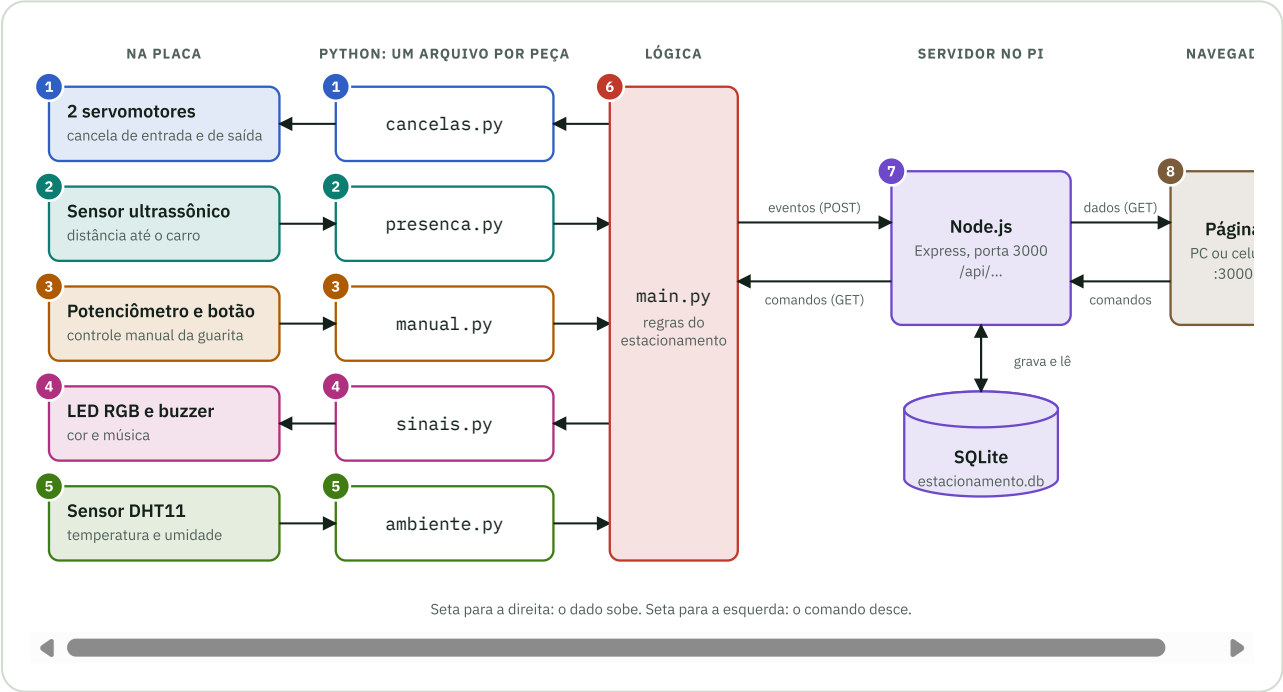
ARQUIVOS DESTA PARTE NO REPOSITÓRIO

ARQUIVO	PARA QUE SERVE
`hardware/placa_interface.kicad_pcb` e `.kicad_pro`	Projeto no KiCad 10
`hardware/montagem_placa_interface.pdf`	4 páginas: lado dos componentes, lado do cobre, etiqueta ampliada e lista de montagem

ARQUIVO	PARA QUE SERVE
`hardware/trilhas_tamanho_real_A4.pdf`	Trilhas em tamanho real (1:1) para conferir sobre a placa
`hardware/etiquetas_A_B_pimaco6282.pdf`	Etiqueta para uma placa. Há também `_folha_cheia` (7 placas) e `_teste_papel_comum`
`hardware/gerador/`	Scripts que geram tudo a partir do `design.py`, a fonte da verdade (peças, furos e trilhas)
`docs/lista_de_materiais.csv` e `ligacoes_netlist.csv`	As tabelas acima, para abrir em planilha

Visão geral do sistema e etapas do trabalho

Como as partes se ligam (clique num bloco para abrir a tarefa) e o que cada tarefa faz em cada etapa



1 Cancelas 2 Presença 3 Guarita manual 4 Sinalização 5 Ambiente
6 Lógica 7 Backend 8 Frontend

ETAPAS DO TRABALHO

Fase 0, todos: entrar no Raspberry e rodar o `rgb_pwm_interativo.py`.

TAREFA	FASE 1: ESTUDAR	FASE 2: ENTREGAR	FASE 3: INTEGRAR COM
1 **Cancelas**	Mover os 2 servos	`cancelas.py`	2: abre quando o carro chega
2 **Presença**	Ler distância, conferir com régua	`presenca.py`	1: carro na frente, cancela abre
3 **Guarita**	Calibrar o pot, ler o botão	`manual.py`	2: o pot move o servo 2 (saída)
4 **Sinalização**	Cores e 4 notas	`sinais.py`	6: um sinal por estado
5 **Ambiente**	Ler temp. e umidade	`ambiente.py`	7: enviar a leitura
6 **Lógica**	Estados e esqueleto com falsos	`main.py` rodando	troca falsos pelos reais
7 **Backend**	API e teste com curl	endpoints do contrato	6: eventos de verdade
8 **Frontend**	Painel com números de exemplo	painel lendo `/api/resumo`	7: gráfico e botão de saída

Fase 4, todos: ensaio e demonstração (roteiro em Demo).

DECISÕES JÁ TOMADAS

Servos
Servo 1 (GPIO13, J2) é a entrada. Servo 2 (GPIO26, J3) é a saída, que o pot controla no modo manual.

Saída
O porteiro clica “Liberar saída” no painel. O servo 2 abre, espera 2 s e fecha. Conta uma saída.

Vagas
`MAX_VAGAS = 7` mora no `server.js` e sai no `/api/resumo` como `vagas`. A cancela fecha 2 s depois do carro passar.

Códigos de teste (antigo)

Os 9 scripts de teste do começo do projeto (pasta `~/codigos`), conferidos com os pinos da placa: 2 estão certos, 1 é só teste e 6 precisam de ajuste. No sistema final, importe os pinos de

`pinos.py` em vez de digitar números.

`rgb_pwm_interativo.py` — Certo

R=4, G=17, B=27, botão=5. Use de exemplo de LED RGB e botão.

`calibracao_pot2.py` — Certo

12 carrega, 25 lê. Base da tarefa 3. Falta salvar a calibração.

`calibracao_pot.py` — Só teste

Pinos 12 e 25 certos. A `pot2` a substituiu.

`sensor_distancia.py` — TRIG e ECHO trocados

Use `DistanceSensor(echo=24, trigger=23, max_distance=2)`. Troque o `LED(2)` pelo LED RGB.

`sensor_distacia_rgb.py` — Pinos e LED errados

Use `RGBLED(red=4, green=17, blue=27, active_high=False)` e os pinos do sensor acima. Renomeie “distacia”.

`led_rgb.py` — Pinos antigos

Troque o `TrafficLights(2, 3, 4)` por `RGBLED` com 4, 17 e 27.

`blink.py` — Pisca invertido

GPIO17 é o verde do RGB (ânodo comum). Use `LED(17, active_high=False)`.

`ledBotao.py` — Botão errado

O botão da placa é o GPIO5: `Button(5)`. O LED, como no `blink.py`.

`potenciometro.py` — Pinos antigos

Usa 14 e 15. Não use: troque pela `calibracao_pot2.py`.

PINOS.PY: A ÚNICA FONTE DE VERDADE DOS PINOS

pinos.py

```
# pinos.py: a única fonte de verdade dos pinos da placa (numeração BCM)

SERVO_ENTRADA = 13   # J2
SERVO_SAIDA = 26     # J3

US_TRIG = 23         # J4, TRIG
US_ECHO = 24         # J4, ECHO (passa pelo divisor R3/R4)

LED_R, LED_G, LED_B = 4, 17, 27   # D1, ânodo comum: nível baixo acende

BUZZER = 6           # J6, via R6 e transistor Q1
BOTAO = 5            # SW1 (e J8 em paralelo)

POT_CARGA = 12       # carrega o capacitor C1 (via R2)
POT_LE = 25          # lê o tempo de carga (via R1 + potenciômetro)

DHT11 = 22           # J5, DAT
```

API: o que as tarefas 5, 6, 7 e 8 combinam

Seis chamadas. Fechem isto no primeiro dia.

DE → PARA	CHAMADA	CORPO OU RESPOSTA
Python → Node	`POST /api/eventos`	`{"tipo": "entrada"}` ou `{"tipo": "saida"}`. Uma saída com ninguém dentro devolve 409
Python → Node	`POST /api/ambiente`	`{"temperatura": 26.0, "umidade": 58}`. Valores que não são números devolvem 400
Python → Node	`POST /api/controlador/batimento`	Sem corpo, responde 204. O `main.py` envia a cada 3 s enquanto espera. Sem ele, depois de 15 s o servidor considera o controlador desligado
Python → Node	`GET /api/comandos/proximo`	O comando mais antigo ainda não feito, já marcado como feito: `{"alvo": "saida", "acao": "abrir"}`. Se não há, 204 sem corpo. Comando que ninguém pegou em 30 s vira expirado

DE → PARA	CHAMADA	CORPO OU RESPOSTA
Página → Node	`GET /api/resumo`	`{"entradas": 42, "saidas": 39, "dentro": 3, "vagas": 7, "controlador_online": true, "por_dia": [{"dia": "2026-11-03", "entradas": 8, "saidas": 7}, ...]}` (últimos 7 dias; dentro e vagas valem para agora; `controlador_online` diz se o `main.py` está ligado)
Página → Node	`GET /api/ambiente/ultima`	`{"temperatura": 26, "umidade": 58, "criado_em": "..."}`
Página → Node	`POST /api/comandos`	`{"alvo": "saida", "acao": "abrir"}`. 201 criado; 200 com `"repetido": true` se já há um pedido pendente; 400 se alvo ou ação estiverem errados; 409 se ninguém está dentro; 503 se o controlador está desligado

Roteiro da demonstração (3 minutos)

É o ensaio da fase 4. Se o time cumprir a lista, o sistema está pronto.

1. **Painel zerado**
   Banco novo, 7 vagas disponíveis.
2. **Carro chega**
   Caixa a menos de 5 cm: amarelo, música, cancela abre.
3. **Carro passa**
   Fecha 2 s depois, verde, as vagas disponíveis diminuem.
4. **Lotado**
   Repita até as vagas disponíveis chegarem a 0: vermelho, 2 bips, não abre.
5. **Saída**
   “Liberar saída” no painel: servo 2 abre, fecha em 2 s, as vagas disponíveis aumentam.
6. **Manual**
   Gire o pot: a cancela de saída acompanha. Aperte o botão: liga e desliga o modo manual (LED azul).
7. **Ambiente e semana**
   Temperatura, umidade e gráfico da semana.

MATERIAL DE APOIO

pinout.xyz: os 40 pinos | gpiozero: receitas | gpiozero: servo, LED RGB, buzzer | gpiozero: botão e ultrassônico | Express: primeiro servidor | better-sqlite3 | Chart.js | Fetch (MDN)

Entrega: o que foi entregue e como usar

O que tem no repositório e como usar cada parte

O QUE TEM NO REPOSITÓRIO

PASTA OU ARQUIVO	O QUE TEM
`codigo/sistema/`	`main.py`, `cancelas.py`, `presenca.py`, `manual.py`, `sinais.py`, `ambiente.py`, `pinos.py`, `server.js`, `public/`, `estacionamento.service` e `estacionamento-web.service`
`codigo/starter/`	Código de partida das 8 tarefas
`codigo/exemplos_originais/`	Os 9 scripts do começo do projeto
`hardware/`	Projeto KiCad, PDFs de montagem, trilhas em 1:1 e etiqueta Pimaco
`docs/`	Lista de materiais e ligações (CSV)
`guia/`	Este guia (`index.html`) e o gerador dele
`README.md`	Resumo do projeto e comandos

COMO USAR

RODAR O SISTEMA NO RASPBERRY

```
# No Raspberry (uma vez)
sudo apt update && sudo apt install -y python3-gpiozero python3-requests libgpiod2 nodejs npm
pip3 install gpiod

# No seu PC: copiar o sistema (troque pelo IP de hoje)
scp -r codigo/sistema dsm@192.168.0.153:~/codigos/final

# No Raspberry: instalar o servidor e ligar os serviços
cd ~/codigos/final && npm install
sudo cp estacionamento.service estacionamento-web.service /etc/systemd/system/
sudo systemctl daemon-reload
```

```
sudo systemctl enable --now estacionamento-web estacionamento

# Painel, no navegador do PC ou do celular
http://raspberrypi.local:3000
```

MONTAR A PLACA

- Lista de compras: `docs/lista_de_materiais.csv`
- Etiqueta (imprimir a 100%): `hardware/etiquetas_A_B_pimaco6282.pdf`
- Ordem de montagem e testes: seção “Replicar a placa” deste guia

TESTAR UMA PEÇA

```
# pare o sistema antes (veja “Rodar sem conflito”)
sudo systemctl stop estacionamento
python3 codigo/starter/tarefa2_teste_distancia.py
```

GERAR DE NOVO ESTE GUIA

```
python3 guia/gerador/montar.py
```

Guia gerado a partir do projeto da placa de interface (versão 2) e dos scripts da pasta `~/codigos`.